Taller de IA para No Informáticos

Guía Completa 2026

Diego Saavedra

Taller de IA para No Informáticos
Manual del Participante
¿Qué aprenderás?
Requisitos Previos
Estructura del Taller
1. Fundamentos: ¿Por qué usar asistentes de IA?
2. El Ecosistema de IA en 2026
3. Instalación de Opencode
4. Tu Primer Contacto con la IA
5. Perfiles Personalizados (agents.md)
6. Casos de Uso Detallados por Profesión
7. Skills y Extensiones
8. MCP y Extensiones
9. Análisis Comparativo
10. Resumen y Próximos Pasos
Referencias Científicas
Preguntas Frecuentes
Cheatsheet
Taller OpenCode para No Informáticos
Guía Completa 2026
📋 Tabla de Contenidos
1. ¿Qué es OpenCode?
2. Instalación
3. Configuración Básica
4. Sistema de Agentes
5. AGENTS.md - Personalización
6. MCP - Extensiones
7. Skills - Habilidades
Avances de la Semana
Próxima Semana
Bloqueantes
Métricas
Flujo de Trabajo Recomendado
1. Objetivos de Comunicación
2. Público Objetivo
3. Competencia
4. Entregables
5. Restricciones
9. Ejercicios Prácticos

📚 Recursos Adicionales
✅ Checklist de Verificación del Taller
🔧 Comandos Rápidos de Referencia
📞 Soporte
Estructura OpenCode - Guía Completa
Arquitectura OpenCode
1. opencode.json - Estructura Completa
2. AGENTS.md - Perfil de Usuario
3. SKILL.md - Estructura de Skills
Soft Rules
Decision Gates
Execution Steps
Output Contract
Referencias
Soft Rules
Decision Gates
Execution Steps
Output Contract
Referencias
6. MCP.json - Template Genérico
7. Estructura de Archivos - Proyecto Completo
8. Permisos - Seguridad
9. Template: Nuevo Agent
10. Template: Nuevo Skill
Soft Rules
Decision Gates
Execution Steps
Output Contract
Referencias
3. Soft Rules (Preferencias)
4. Decision Gates
5. Execution Steps
6. Output Contract
7. Code Quality Checklist
8. References
9. Stack-Specific Rules
10. Error Handling Protocol
11. Testing Requirements
3. Soft Rules (Preferencias)
4. Decision Gates
5. Course Structure Template
Assessment Strategy
8. Output Contract
9. ABACOM Compliance Checklist
10. References
3. Soft Rules (Preferencias)
4. Decision Gates
5. PTES Methodology Phases
6. Bug Bounty Workflow
Vulnerability Testing Checklist
Severity Classification
9. Responsible Disclosure Template
10. References
LAB 2: Configuración de Perfil Personal (20 minutos)

LAB 3: Tu Primer Agente Especializado (25 minutos)
Resumen
Contenido principal
Conclusión
LAB 5: Caso Práctico End-to-End (30 minutos)
Lab Bonus: Automatización con Skills
Resumen de Labs
Recursos Adicionales

Taller de IA para No Informáticos

Manual del Participante

Duración: 2 Horas | Modalidad: Virtual | Costo: Gratis

¿Qué aprenderás?

Al finalizar este taller podrás usar herramientas de inteligencia artificial para aumentar tu productividad profesional sin necesidad de saber programar. Configurarás tu propio asistente personalizado y entenderás qué herramienta usar según tu trabajo.

Al terminar serás capaz de:

- Usar Opencode para tareas profesionales
- Personalizar tu asistente con tu perfil
- Instalar habilidades adicionales
- Conectar extensiones
- Elegir la mejor herramienta para tu trabajo

Requisitos Previos

Antes del taller prepara:

1. Cuenta de Google (para NotebookLM, gratis)
2. Editor de texto (Bloc de notas o TextEdit)
3. Navegador actualizado (Chrome, Firefox o Edge)
4. Conexión a internet estable

Estructura del Taller

Hora 1: Fundamentos y Configuración

Tiempo	Tema
0:00 - 0:15	Bienvenida e introducción
0:15 - 0:30	El ecosistema de IA en 2026
0:30 - 0:45	Instalación de Opencode
0:45 - 1:00	Primera conversación con la IA

Hora 2: Personalización y Análisis

Tiempo	Tema
1:00 - 1:20	Perfiles personalizados (agents.md)
1:20 - 1:40	Skills y extensiones
1:40 - 1:55	Comparativa de herramientas
1:55 - 2:00	Cierre y siguientes pasos

---

1. Fundamentos: ¿Por qué usar asistentes de IA?

Evidencia científica

La investigación científica demuestra beneficios significativos:

Estudio de Repsol (2024)

La multienergética española, en colaboración con MIT Technology Review, evaluó Microsoft 365 Copilot durante 4 meses con 300 empleados:

- **Ahorro de tiempo**: 121 minutos brutos por semana por empleado (96.8 horas/año)
- **Mejora en calidad**: 16% increase en la calidad de entregables
- **Satisfacción**: 61.9% declaró que preferiría no volver a trabajar sin la herramienta
- **Perfil más benefici**: Técnicos cualificados y profesionales sénior

Estudio en Ecuador (El Oro, 2025)

Investigación en 900 empleados de empresas comerciales:

- **Productividad**: El uso frecuente de IA incrementa la productividad (odds ratio de 2.34)
- **Eficiencia**: 40% de reducción en duración de tareas administrativas
- **Precisión**: 72% de reducción en incidencia de errores

Estudio en México (2025)

Encuesta a 327 usuarios mexicanos:

- **Productividad**: 62% usa IA para incrementar productividad
- **Ahorro**: 7.02 horas semanales en promedio
- **Creatividad**: 31.5% usa IA para generar nuevas ideas
- **Motivación**: Cerca de un tercio busca innovación

Estudio en Universidades Latinoamericanas (2024)

Implementación de asistentes virtuales en universidades de Ecuador, Perú y Colombia:

- **Satisfacción**: Más del 75% reportó altos niveles de satisfacción
- **Docentes**: Reducción de carga administrativa
- **Estudiantes**: Retroalimentación inmediata y adaptación de contenidos

Conclusión científica

Los estudios demuestran consistentemente que los asistentes de IA:

1. **Ahorran tiempo significativo** (2+ horas/semana)
2. **Mejoran la calidad** del trabajo realizado
3. **Aumentan la satisfacción** laboral
4. **Reducen errores** en tareas repetitivas

La IA no reemplaza al profesional, pero amplifica su capacidad.

---

2. El Ecosistema de IA en 2026

¿Qué es un asistente de IA?

Es como un compañero virtual que te ayuda con tareas intelectuales: escribir, investigar, analizar, resumir. A diferencia de los motores de búsqueda, entiende el contexto y puede mantener una conversación.

Las principales herramientas

Herramienta	Para qué sirve	Ventaja principal
Opencode	Asistencia versatile	Memoria persistente y personalización
Copilot	Programación	Integrado en VS Code
Gemini	Ecosistema Google	Integrado con Sheets, Docs, Drive
Ollama	Modelos locales	Trabaja sin internet
NotebookLM	Análisis de documentos	

Herramienta	Para qué sirve	Ventaja principal
		Generación de contenido desde fuentes

¿Cuál elegir?

Si necesitas…	Elige…
Flexibilidad sin programar	Opencode
Programar en VS Code	Copilot
Usar el ecosistema Google	Gemini
Privacidad total de datos	Ollama
Solo analizar documentos	NotebookLM

---

3. Instalación de Opencode

¿Por qué Opencode?

Framework de asistencia versatile con:

- **Memoria persistente** (recuerda conversaciones anteriores)
- **Sistema de perfiles** (se adapta a tu trabajo)
- **Skills modulares** (extiende capacidades)
- **MCPs** (se conecta con servicios externos)

Instalación

Windows: Abre PowerShell y escribe:

```
npm install -g opencode-cli
```

Mac: Abre Terminal:

```
npm install -g opencode-cli
```

Linux:

```
sudo npm install -g opencode-cli
```

Verifica instalación

```
opencode --version
```

---

4. Tu Primer Contacto con la IA

Iniciar conversación

```
opencode
```

Ejemplos de conversación

Presenter:

```
Hola, me llamo [tu nombre] y trabajo en [tu industria]. Ayúdame
con [tu tarea].
```

Pedir ayuda:

```
Tengo que escribir un proyecto sobre [tema]. ¿Puedes ayudarme a
estructurarlo?
```

Investigar:

```
Busca información sobre [tu tema de interés]
```

---

5. Perfiles Personalizados (agents.md)

¿Qué es agents.md?

Es una tarjeta de presentación para tu asistente. Le dice quién eres, qué necesitas y cómo prefieres trabajar.

Estructura básica

```md
# Mi Perfil Profesional

## Quién soy
[Nombre]
[Profesión/Rol]
[Industria]

## Qué hago
[Descripción]

## Mis objetivos
- Objetivo 1
- Objetivo 2

## Cómo prefiero que me ayudes
[Estilo preferido]
[Nivel de detalle]
```

Ejemplo: Educator

```
# Mi Perfil Profesional

## Quién soy
María García
Profesora de matemáticas
Educación secundaria

## Qué hago
Clases de bachillerato.
Evaluaciones.
Tutorías.

## Mis objetivos
- Crear planes de clase efectivos
- Generar evaluaciones equilibradas
```

Ejemplo: Administrador

```
# Mi Perfil Profesional

## Quién soy
Juan Pérez
Gerente de operaciones
Sector: Retail

## Qué hago
Coordinación logística.
Gestión de inventario.
Reportes semanales.

## Mis objetivos
- Automatizar reportes
- Mejorar análisis
```

---

6. Casos de Uso Detallados por Profesión

Caso 1: Educator / Docente

Perfil: Profesora de historia enbachillerato

Necesidad: Crear plan de unidad sobre la Edad Media

Con Opencode:

1. Configura tu perfil de educator
2. Escribe: “Crea un plan de unidad sobre la Edad Media para 3er año”
3. Recibes estructura con objetivos, contenidos, actividades
4. Pide: “Genera una evaluación con 10 preguntas”

5. Recibes preguntas con rúbrica de corrección
6. Pide: “Busca fuentes actualizadas sobre [tema específico]”

Resultado: Plan de unidad completo en 15 minutos vs 3 horas manual

Herramientas útiles:

- Opencode para generación de contenido
- NotebookLM para analizar recursos educativos

Caso 2: Administrador de Empresas

Perfil: Gerente de recursos humanos

Necesidad: Elaborar informe de rotación de personal

Con Opencode:

1. Configura tu perfil de RRHH
2. Escribe: “Ayúdame a estructurar un informe de rotación de personal”
3. Recibes plantilla con métricas clave
4. Proporciona tus datos
5. Pide: “Análisis de las causas principales”
6. Recibes recomendaciones basadas en datos

Resultado: informe profesional con insights accionables

Herramientas:

- Opencode para estructuración
- Gemini para análisis de hojas de cálculo

Caso 3: Profesional de Marketing

Perfil: Coordinador de marketing digital

Necesidad: Calendario de contenido para redes sociales

Con Opencode:

1. Configura perfil de marketing
2. Escribe: “Crea un calendario de contenido para Instagram para el próximo mes”
3. Recibes 30 publicaciones con temas, horarios, hashtags
4. Pide: “Analiza las redes de [competidor]”
5. Recibes análisis de competidores
6. Pide: “Genera captions para 5 publicaciones”

Resultado: Contenido para un mes en 30 minutos

Herramientas:

- Opencode para generación

- Amp para automatización

Caso 4: Profesional de Salud

Perfil: Médico general en centro de salud

Necesidad: Mantenerse actualizado en protocolos

Con Ollama + NotebookLM:

1. Configura Ollama (trabajo offline)
2. Busca: "Protocolos actualizados diabetes tipo 2 2025"
3. En NotebookLM: Agrega artículos de revistas médicas
4. Pide: "Resume los hallazgos principales"
5. Verifica siempre con fuentes primarias

Privacidad: Datos de pacientes nunca dejan la institución

Herramientas:

- Ollama (privacidad)
- NotebookLM (análisis de papers)

Caso 5: Abogado / Legal

Perfil: Lawyer en bufete median

Necesidad: Investigar precedentes judiciales

Con Opencode:

1. Configura perfil legal
2. Escribe: "Busca precedentes sobre [tema jurídico]"
3. Recibes casos relevantes con citas
4. Pide: "Resume los argumentos principales"
5. Verifica siempre con fuentes oficiales

Herramientas:

- Opencode para búsqueda
- Normas de citación profesional

Caso 6: Contador / Contador Público

Perfil: Contador en empresa comercial

Necesidad: Entender nuevas regulaciones fiscales

Con Opencode:

1. Configura perfil contable

2. Escribe: “Explica las reformas tributarias 2025”
3. Recibes resumen en lenguaje claro
4. Pide: “¿Cómo afecta a empresas del sector retail?”
5. Recibes análisis específico

Herramientas:

- Opencode para explicación
- Gemini para hojas de cálculo

Caso 7: Periodista / Comunicador

Perfil: Journalist de investigación

Necedad: Investigar tema complejo

Con Opencode:

1. Configura perfil de periodista
2. Escribe: “Investiga sobre [tema]”
3. Recibes fontes confiables
4. Pide: “Resume en 5 oraciones”
5. Pide: “¿Qué ángulos no han sido explorados?”

Herramientas:

- Opencode para investigación
- NotebookLM para analizar documentos

Caso 8: Ingeniero (No de Software)

Perfil: Civil engineer en constructora

Necesidad: Revisar normativas de construcción

Con Opencode:

1. Configura perfil de ingeniería civil
2. Escribe: “¿Cuáles son los requisitos sísmicos para跳水 de concreto?”
3. Recibes norma con parámetros
4. Pide: “Busca ejemplos de aplicación”
5. Recibes casos de estudio

Herramientas:

- Opencode para normativa
- Búsqueda de estándares

Caso 9: Arquitecto

Perfil: Arquitecto en estudio de diseño

Necesidad: Investigar tendencias sostenibles

Con Opencode:

1. Configura perfil de arquitectura
2. Escribe: “Tendencias en materiales sostenibles 2025”
3. Recibes lista de materiales con propiedades
4. Pide: “Proyectos Beispiel que los usen”
5. Recibes casos reales con fotos

Caso 10: Investigador Académico

Perfil: Profesor investigador universitario

Necesidad: Revisión bibliográfica para artículo

Con Opencode:

1. Configura perfil de investigador
2. Escribe: “/busqueda_cientifica [tu tema]”
3. Recibes 20+ fuentes Q1/Q2
4. Verifica en NotebookLM
5. Pide: “Resume el estado del arte”
6. Solicita: “Genera debate entre autores”

Resultado: Revisión de literatura completa en horas vs semanas

---

Caso 11: Estudiante Universitario

Perfil: Estudiante de maestría

Necesidad: Completar trabajo de investigación

Con Opencode:

1. Configura tu perfil de estudiante
2. Escribe: “Ayúdame a estructurar mi TFM sobre [tema]”
3. Recibes estructura IMRAD
4. Proporciona avances
5. Pide: “Revisa mi redacción”
6. Recibes correcciones

Herramientas:

- Opencode para estructura
- NotebookLM para estudiar

Caso 12: Emprendedor / Pequeño empresario

Perfil: Owner de tienda online

Necedad: Mejorar rentabilidad

Con Opencode:

1. Configura perfil de negocio
2. Escribe: “Analiza mis ventas del último mes [comparte datos]”
3. Recibes análisis de patrones
4. Pide: “Recomienda productos a ordenar”
5. Pide: “Estrategia de precios”

---

7. Skills y Extensiones

¿Qué son las skills?

Aplicaciones adicionales queextienden las capacidades de Opencode.

Skill	Para qué sirve
Escritura académica	Informes, documentos
Marketing digital	Estrategias, contenido
Investigación	Fuentes confiables
Análisis de datos	Hojas de cálculo
Presentaciones	Diapositivas
NotebookLM	Análisis profundo

Cómo usar

Simplemente indícale qué necesitas:

```
Necesito ayuda con [tu tarea]
```

---

8. MCP y Extensiones

¿Qué es MCP?

Protocolo que permite conexiones con herramientas externas.

MCPs esenciales:

MCP	Función
NotebookLM	Análisis de documentos

MCP	Función
Context7	Documentación
Engram	Memoria persistente

---

9. Análisis Comparativo

Comparativa principal

Característica	Copilot	Opencode	Ollama
Costo	Freemium	Freemium	Gratis
Curva	Media	Baja	Media
Memoria	No	Sí	Sí
Personalización	Limitada	Profunda	Media
Internet	Sí	Sí	No
Datos locales	No	No	Sí

Por profesión

Profesión	Recomendación
Educator	Opencode + NotebookLM
Administrador	Opencode
Marketing	Opencode + Amp
Salud	Ollama (privacidad)
Legal	Opencode
Contador	Opencode + Gemini

---

10. Resumen y Próximos Pasos

Lo que aprendiste

1. **Evidencia científica**: Los estudios demuestran beneficios reales
2. **Ecosistema IA**: Herramientas disponibles
3. **Opencode**: Instalación básica
4. **Perfiles**: Personalización con agents.md
5. **Skills**: Extensiones de capacidades
6. **Casos de uso**: 12 profesiones cubiertas

Plan post-taller

Semana	Acción
1ra	Personaliza tu agents.md
2da	Explora 2-3 skills
3ra	Integra en tu flujo
4ta	Comparte con colegas

Referencias Científicas

1. **Repsol (2024)**. Estudio sobre Microsoft 365 Copilot. MIT Technology Review España.
2. Márquez-Romero et al. (2025). Impacto de asistentes de IA en productividad. Revista de Finanzas (Ecuador).
3. Yaranga & Olortiga (2025). IA y productividad empresarial. Estudio bibliométrico.
4. López et al. (2025). Asistentes virtuales en educación superior. Revista EDU-INNOVA.
5. Jardón et al. (2024). Asistentes IA en universidades latinoamericanas.
6. Hidalgo Toledo (2025). Productividad y creatividad con IA en México. SciELO.

Preguntas Frecuentes

¿Necesito saber programar? No, el taller es para profesionales sin conocimientos técnicos.

¿Cuánto cuesta? Opencode tiene nivel gratuito. Ollama y NotebookLM son gratuitos.

¿Es seguro? Para datos sensibles usa Ollama (offline). Para datos generales, las herramientas estándar son seguras.

Cheatsheet

```
# Instalación
npm install -g opencode-cli

# Verificar
opencode --version

# Iniciar
opencode

# Skills
```

```
/skills

# Ayuda
ayúdame con [tarea]
```

Material preparado para el Taller de IA para No Informáticos 2 Horas — Virtual — Gratis

Taller OpenCode para No Informáticos

Guía Completa 2026

> **Objetivo**: Capacitar a profesionales no técnicos en el uso de OpenCode como asistente de IA para potenciar su productividad diaria.

📋 Tabla de Contenidos

1. ¿Qué es OpenCode?
2. Instalación
3. Configuración Básica
4. Sistema de Agentes
5. AGENTS.md - Personalización
6. MCP - Extensiones
7. Skills - Habilidades
8. Casos de Uso por Profesión
9. Ejercicios Prácticos

---

1. ¿Qué es OpenCode?

Definición Simple

OpenCode es un **asistente de inteligencia artificial de código abierto** que funciona en tu terminal, escritorio o IDE. A diferencia de ChatGPT o Claude Chat, OpenCode está diseñado específicamente para:

- ✅ **Trabajar con archivos y código** en tu computadora
- ✅ **Ejecutar comandos** del sistema
- ✅ **Conectarse a herramientas externas** (GitHub, Jira, Notion)
- ✅ **Recordar contexto** de tus proyectos
- ✅ **Automatizar tareas repetitivas**

Estadísticas (2026)

Métrica	Valor
GitHub Stars	150,000+
Desarrolladores mensuales	6.5 millones
Contribuidores	850+
Proveedores de modelos IA	75+

¿Por qué es Relevante para No Informáticos?

Aunque su nombre dice "code" (código), OpenCode puede ayudarte con:

- 📝 **Redactar documentos** profesionales
- 📊 **Analizar datos** en hojas de cálculo
- ✉️ **Redactar correos** y mensajes
- 📋 **Gestionar proyectos** (integración con Jira, Notion)
- 🔍 **Investigar** temas en la web
- 📁 **Organizar archivos** automáticamente

---

2. Instalación

Método 1: Script de Instalación (Recomendado)

Este es el método más rápido y funciona en **todos los sistemas operativos**.

Paso a Paso

```
# 1. Abrir la terminal
# En macOS: Cmd + Espacio → "Terminal"
# En Windows: Win + X → "Terminal" (o usar WSL)
# En Linux: Ctrl + Alt + T

# 2. Ejecutar el script de instalación
curl -fsSL https://opencode.ai/install | bash

# 3. Verificar la instalación
opencode --version
```

Opciones de Personalización

```
# Instalar en un directorio personalizado
OPENCODE_INSTALL_DIR=/usr/local/bin curl -fsSL https://
        opencode.ai/install | bash
```

```
# Usar directorio XDG estándar
XDG_BIN_DIR=$HOME/.local/bin curl -fsSL https://opencode.ai/
        install | bash
```

Método 2: npm (Gestor de Paquetes Node.js)

```
# Requiere Node.js 18+ instalado
npm install -g opencode-ai

# O usando otros gestores
bun add -g opencode-ai    # Bun
pnpm add -g opencode-ai   # pnpm
yarn global add opencode-ai # Yarn
```

Método 3: Homebrew (macOS/Linux)

```
# Añadir el tap oficial
brew tap anomalyco/tap/opencode

# Instalar
brew install opencode

# Verificar
opencode --version
```

Método 4: Aplicación de Escritorio (Beta)

Para quienes prefieren **no usar la terminal**:

macOS

```
# Usando Homebrew
brew install --cask opencode-desktop
```

Windows

Descarga directa desde: opencode.ai/download

Linux

- **.deb** para Debian/Ubuntu
- **.rpm** para Fedora/RHEL
- **AppImage** para distribuciones universales

Método 5: Scoop (Windows)

```
scoop install opencode
```

Comparativa de Métodos

Método	Dificultad	Recomendado para
Script curl	⭐ Fácil	90% de usuarios
npm	⭐⭐ Media	Desarrolladores
Homebrew	⭐ Fácil	macOS/Linux
Desktop App	⭐ Fácil	No técnicos
Scoop	⭐⭐ Media	Windows power users

---

3. Configuración Básica

Estructura de Archivos de Configuración

```
MiCarpetaProyecto/
├── opencode.json           # Configuración del proyecto
├── AGENTS.md               # Reglas específicas del proyecto
└── .opencode/
    ├── agents/             # Agentes personalizados
    │   └── mi-agente.md
    ├── skills/             # Habilidades personalizadas
    │   └── mi-skill/
    │       └── SKILL.md
    └── commands/           # Comandos personalizados
        └── mi-comando.md

~/.config/opencode/         # Configuración global del usuario
├── opencode.json
├── AGENTS.md
├── agents/
├── skills/
└── commands/
```

Configuración Inicial con `/init`

```
# Ejecutar dentro de tu carpeta de proyecto
opencode /init

# Esto analizará tu proyecto y creará un AGENTS.md automáticamente
```

Configurar tu Primera API Key

Opción A: Variables de Entorno (Recomendado)

```
# Añadir a ~/.zshrc (macOS/Linux)
echo 'export OPENAI_API_KEY="sk-tu-clave-aqui"' >> ~/.zshrc
source ~/.zshrc
```

```bash
# Añadir a ~/.bashrc (Linux)
echo 'export OPENAI_API_KEY="sk-tu-clave-aqui"' >> ~/.bashrc
source ~/.bashrc
```

Opción B: Usando el Comando `/connect`

```bash
opencode
# Dentro de OpenCode ejecutar:
/connect
# Seleccionar proveedor (OpenAI, Anthropic, etc.)
# Seguir instrucciones de autenticación
```

opencode.json Mínimo

Crea `~/.config/opencode/opencode.json`:

```json
{
  "$schema": "https://opencode.ai/config.json",
  "provider": "openai",
  "model": "gpt-4o",
  "theme": "opencode",
  "permission": {
    "edit": "ask",
    "bash": "ask",
    "webfetch": "allow"
  }
}
```

---

4. Sistema de Agentes

¿Qué es un Agente?

Un **agente** es una configuración específica que define:

- 🎯 **Qué puede hacer** el asistente
- 🔒 **Qué permisos tiene** (leer, escribir, ejecutar comandos)
- 🧠 **Cómo piensa** (prompt personalizado)
- 🛠️ **Qué herramientas usa** (MCP, skills)

Agentes Integrados

OpenCode viene con agentes predefinidos:

Agente	Propósito	Permisos
`@plan`	Planificación y análisis (NO hace cambios)	Solo lectura
`@code`	Implementación de código	Lectura + Escritura

Agente	Propósito	Permisos
`@explore`	Explorar y entender proyectos	Solo lectura
`@scout`	Investigar documentación externa	Web
`@general`	Tareas generales	Todo

Usar un Agente Específico

```
# Ejecutar con un agente específico
opencode @plan "Analiza la estructura de este proyecto"

# O dentro de OpenCode
@plan Analiza la estructura de este proyecto
```

Crear un Agente Personalizado

Paso 1: Crear el archivo del agente

Crea `~/.config/opencode/agents/redactor.md`:

```
---
description: Asistente de redacción profesional
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: allow
---

# Eres un redactor profesional

Tu rol es ayudar con:
- Escritura de correos electrónicos formales
- Redacción de informes y documentos
- Revisión gramatical y ortográfica
- Estilo profesional latinoamerican

## Reglas de Escritura

1. Usar español latinoamerican formal
2. Tono respetuoso pero cercano
3. Oraciones claras y directas
4. Evitar anglicismos innecesarios
5. Incluir frases de transición

## Formato de Documentos

- Títulos: Mayúsculas iniciales
- Listas: Guiones (no números)
- Cierre: "Quedo atento/a a sus comentarios"
```

Paso 2: Usar el agente

```
@redactor Redacta un correo formal solicitando una reunión de
        seguimiento
```

Ejemplo: Agente para Marketing

~/.config/opencode/agents/marketing.md:

```md
---
description: Asistente de marketing y redes sociales
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: allow
---

# Eres un consultor de marketing digital

Experto en:
- Marketing digital para Latinoamérica
- Redes sociales (Instagram, LinkedIn, TikTok)
- Email marketing
- Creación de contenido

## Voz de Marca

- Tono: Profesional pero accesible
- Idioma: Español latinoamerican
- Formato: Adaptable a cada plataforma

## Plataforma-Específico

### LinkedIn
- Posts de 150-300 palabras
- Incluir call-to-action
- Usar emojis con moderación

### Instagram
- Captions de 125-150 caracteres
- Hashtags relevantes (#negocios, #emprendimiento)
- Incluir línea de acción

### Correos
- Asunto: Claro y conciso
- Cuerpo: Máximo 3 párrafos
- CTA visible
```

---

5. AGENTS.md - Personalización

¿Qué es AGENTS.md?

`AGENTS.md` es un archivo especial que le dice a OpenCode **cómo comportarse** en tu proyecto o de forma global.

Ubicaciones y Prioridad

```
1. ./AGENTS.md                    → Reglas del proyecto actual
(PRIORIDAD ALTA)
2. ./packages/*/AGENTS.md        → Reglas por paquete
3. ~/.config/opencode/AGENTS.md   → Reglas globales del usuario
(MEDIA)
4. ~/.claude/CLAUDE.md           → Compatibilidad con Claude Code
(BAJA)
```

> **Nota**: Si existe `AGENTS.md`, se ignora `CLAUDE.md`. Si existe `~/.config/opencode/AGENTS.md`, se ignora `~/.claude/CLAUDE.md`.

Ejemplo Completo para No Técnicos

Proyecto: Gestión de Redes Sociales

Crea `AGENTS.md` en tu proyecto:

```
# Proyecto: Gestión de Redes Sociales - Empresa XYZ

## Contexto del Negocio

Somos una pyme ecuatoriana dedicada a [describir negocio].
Nuestra audiencia objetivo son emprendedores y pequeños negocios.
Presencia en: Instagram, LinkedIn, Facebook.

## Estilo de Comunicación

- Tono: Profesional pero cercano, inspira confianza
- Idioma: Español latinoamerican (Ecuador)
--No usar: Argot excesivo, anglicismos innecesarios
- Sí usar: Frases cortas, verbos de acción, números específicos

## Tipos de Contenido

1. Educativo: Tips y consejos prácticos
2. Inspiracional: Frases motivacionales del día
3. Promocional: Productos/servicios destacados
4. Interactivo: Encuestas, preguntas a la audiencia

## Calendario de Publicación

- Lunes: Tip de la semana
```

```
- Miércoles: Producto destacado
- Viernes: Contenido motivacional
- Sábados: Interactivo (stories)

## Hashtags por Categoría

### Negocio
#emprendimiento #pyme #negocios #empresa # Ecuador

### Motivación
#lunesdemotivación #exito #logros #crecimiento

### Producto
#[nombredemarca] #[categoría]

## Restricciones Importantes

- NO publicar contenido político o religioso
- NO usar imágenes con texto excesivo
- SIEMPRE incluir alt text en imágenes
- VERIFICAR horarios: 9:00-11:00 y 18:00-20:00 (hora Ecuador)

## Proceso de Aprobación

Antes de publicar:
1. Revisión de ortografía
2. Verificar hashtags (máximo 15)
3. Adjuntar imagen/recurso
4. Esperar confirmación antes de publicar
```

Ejemplo: Agente de Atendimento al Cliente

```
# Sistema de Atención al Cliente - Soporte Técnico

## Tu Rol

Eres un agente de soporte técnico especializado.
Tu objetivo es resolver problemas de forma eficiente y amable.

## Principios de Atención

1. **Escuchar primero**: Entender el problema antes de solución
2. **Ser empático**: "Entiendo su frustración"
3. **Ser claro**: Explicaciones simples, sin jerga técnica
4. **Ser paciente**: Preguntar hasta entender completamente

## Tipos de Consultas y Respuestas

### Consulta de Horarios
" Nuestros horarios de atención son:
- Lunes a Viernes: 8:00 a 18:00
- Sábados: 9:00 a 13:00
```

```
- Domingos: Cerrado

¿Hay algo más en lo que pueda ayudarle?"

### Consulta de Precios
"A continuación le presento nuestros precios:
[Producto A]: $XX.XX
[Producto B]: $XX.XX

¿Le gustaría más información sobre algún producto?"

### Consulta Técnica
"Para resolver su problema, necesito verificar algunos datos:
1. ¿Qué dispositivo está usando?
2. ¿Desde cuándo ocurre el problema?
3. ¿Ha intentado algún paso de solución?

Mientras tanto, puede consultar nuestra guía de problemas comunes
        en [enlace]."

## Firmas de Cierre

"¿Hay algo más en lo que pueda ayudarle?"
"Quedo atento/a a sus comentarios"
"Fue un placer atenderle"
```

Configurar Instrucciones Externas

Para proyectos grandes, puedes separar las reglas en múltiples archivos.

`opencode.json`:

```
{
  "$schema": "https://opencode.ai/config.json",
  "instructions": [
    "docs/estilo-comunicacion.md",
    "docs/procesos.md",
    "~/mis-reglas/tono-redes.md"
  ]
}
```

---

6. MCP - Extensiones

¿Qué es MCP?

MCP (Model Context Protocol) es como un **cable USB para IA**. Permite que OpenCode se conecte a herramientas externas como:

- 📁 Google Drive / OneDrive

- 📋 Notion / Jira / Trello
- 📧 Gmail / Outlook
- 🐙 GitHub / GitLab
- 🗄️ Bases de datos
- 🌐 Navegadores web

Instalar un Servidor MCP

Paso a Paso con `opencode mcp add`

```
opencode mcp add
# Seguir las instrucciones interactivas
```

MCPs Recomendados para No Técnicos

1. GitHub MCP (Para gestionar repositorios)

```
# Configuración
opencode mcp add
# Nombre: github
# Tipo: Local
# Comando: npx
# Args: -y @modelcontextprotocol/server-github
# Variables: GITHUB_PERSONAL_ACCESS_TOKEN=tu_token

// En opencode.json
{
  "mcp": {
    "github": {
      "type": "local",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "environment": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{file:~/.config/
        opencode/.secrets/github-token}"
      }
    }
  }
}
```

2. Notion MCP (Gestión de notas y documentos)

```
# 1. Crear integración en notion.so/my-integrations
# 2. Compartir las páginas con la integración
# 3. Configurar:
opencode mcp add
# Nombre: notion
# Comando: npx
# Args: -y notion-mcp-server
# Variables: NOTION_API_KEY=secret_tu_key
```

```
{
  "mcp": {
    "notion": {
      "type": "local",
      "command": "npx",
      "args": ["-y", "notion-mcp-server"],
      "environment": {
        "NOTION_API_KEY": "secret_tu_api_key"
      }
    }
  }
}
```

3. Gmail MCP (Gestión de correo)

```
# Requiere OAuth de Google
opencode mcp add
# Nombre: gmail
# Comando: npx
# Args: -y @modelcontextprotocol/server-gmail
```

4. Playwright MCP (Automatización de navegador)

```
# Para automatizar tareas web
opencode mcp add
# Nombre: browser
# Comando: npx
# Args: -y @modelcontextprotocol/server-playwright
```

5. Slack MCP (Comunicación de equipo)

```
# Integración con Slack
opencode mcp add
# Nombre: slack
# Comando: npx
# Args: -y @modelcontextprotocol/server-slack
# Variables: SLACK_BOT_TOKEN=xoxb-tu-token
```

MCPs Populares Adicionales

MCP	Propósito	Dificultad
Context7	Buscar documentación de librerías	⭐ Fácil
Sentry	Monitoreo de errores	⭐⭐ Media
Figma	Revisión de diseños	⭐ Fácil
n8n	Automatización de workflows	⭐⭐ Media
Filesystem	Acceso a archivos local	⭐ Fácil
PostgreSQL	Consultas a bases de datos	⭐⭐⭐ Alta

Ejemplo: Configurar Context7 (Documentación)

```
{
  "mcp": {
    "context7": {
      "type": "local",
      "command": "npx",
      "args": ["-y", "@context7/mcp-server"]
    }
  }
}
```

Permisos por Agente para MCP

```
{
  "agent": {
    "redactor": {
      "permission": {
        "tool": {
          "notion_*": "allow",
          "gmail_*": "allow",
          "github_*": "deny"
        }
      }
    }
  }
}
```

---

7. Skills - Habilidades

¿Qué es un Skill?

Un **skill** es un conjunto de instrucciones reutilizables que el agente puede cargar cuando lo necesite. Es como un "plugin" de comportamiento.

Estructura de un Skill

```
.opencode/skills/
└── nombre-del-skill/
    └── SKILL.md          # Obligatorio
    ├── archivo1.md       # Opcional
    └── archivo2.md       # Opcional
```

SKILL.md - Formato Obligatorio

```
---
name: nombre-del-skill
```

```yaml
description: Descripción clara de qué hace este skill (1-1024
        caracteres)
license: MIT              # Opcional
compatibility: opencode   # Opcional
metadata:                 # Opcional
  audiencia: desarrolladores
  categoria: frontend
---
```

```markdown
# Contenido del Skill

Aquí van las instrucciones detalladas...
```

Reglas de Nomenclatura

Regla	Correcto	Incorrecto
Minúsculas	`mi-skill`	`MiSkill`
Guiones simples	`redaccion-pro`	`redaccion__pro`
Sin guiones al inicio/final	`mi-skill`	`-mi-skill`
Máximo 64 caracteres	✓	×

Ejemplo de Skill: Redacción de Correos

`.opencode/skills/redaccion-correos/SKILL.md`:

```yaml
---
name: redaccion-correos
description: Plantillas y guía para redactar correos profesionales
        efectivos
compatibility: opencode
metadata:
  idioma: español-latino
  tono: profesional
---
```

```markdown
# Skill: Redacción de Correos Profesionales

## Estructura Obligatoria

Todo correo debe seguir esta estructura:

1. **Saludo** (1 línea)
2. **Cuerpo** (máximo 3 párrafos)
3. **Cierre** (1 línea)
4. **Firma** (nombre + cargo + contacto)

## Plantillas

### Solicitud de Reunión
```

Asunto: Solicitud de reunión - [Tema]

Estimado/a [Nombre],

Me dirijo a usted para solicitar una reunión con el fin de discutir [tema].

¿Sería posible coordinar una reunión [fecha/hora]? Quedo a su disposición para adaptarme a su agenda.

Quedo atento/a a su confirmación.

Saludos cordiales, [Tu nombre] [Cargo] [Contacto]

Respuesta a Queja

Asunto: Re: [Asunto original]

Estimado/a [Nombre],

Agradezco profundamente su Retroalimentación. Tomamos muy en serio este tipo de comentarios.

[Explicar acciones tomadas o que se tomarán]

¿Hay algo adicional que pueda hacer para resolver esta situación?

Atentamente, [Tu nombre]

Seguimiento

Asunto: Seguimiento - [Asunto previo]

Estimado/a [Nombre],

Espero que se encuentre bien. Me permito dar seguimiento a nuestra conversación del [fecha].

[Estado actual o próxima acción]

Quedo a su disposición para cualquier duda.

Saludos, [Tu nombre]

Frases de Cierre

- "Quedo atento/a a sus comentarios"
- "A la espera de su confirmación"
- "Será un placer ampliar la información"
- "Agradezco de antemano su atención"

Errores Comunes a Evitar

❌ "Hola buenos días" (genérico)

```
✅ "Buenos días, estimado Juan"

❌ "Le escribo para..." (directo pero poco cálido)
✅ "Espero que se encuentre bien. Le escribo porque..."

❌ "No dude en contactarme" (pasivo)
✅ "Estoy a su disposición en [canal específico]"
```

Ejemplo de Skill: Gestión de Proyectos

`.opencode/skills/gestion-proyectos/SKILL.md`:

```
---
name: gestion-proyectos
description: Metodología y plantillas para gestión de proyectos
        ágiles
compatibility: opencode
metadata:
  metodologia: agile
  region: latinoamerica
---

# Skill: Gestión de Proyectos Ágiles

## Principios Fundamentales

1. **Iteraciones cortas**: 1-2 semanas
2. **Entregas frecuentes**: Valor incremental
3. **Retroalimentación**: Del cliente y del equipo
4. **Adaptación**: Cambiar según necesidad

## Estructura de Historia de Usuario

COMO: [tipo de usuario] QUIERO: [acción específica] PARA: [beneficio/valor]


### Ejemplo

COMO: Jefe de ventas QUIERO: Ver un dashboard con métricas semanales PARA:
Tomar decisiones informadas rápidamente


## Criterios de Aceptación

Toda historia debe tener:

DADO QUE: [contexto previo] CUANDO: [acción del usuario] ENTONCES:
[resultado esperado]


## Tipos de Reuniones

### Daily Standup (15 min)
- ¿Qué hice ayer?
```

```
- ¿Qué haré hoy?
- ¿Qué me bloquea?

### Sprint Planning (1-2 hrs)
1. Seleccionar historias del backlog
2. Estimar esfuerzo (puntos de historia)
3. Asignar responsable

### Sprint Review (1 hr)
1. Demostrar funcionalidades completadas
2. Recoger Retroalimentación
3. Actualizar backlog

### Retrospectiva (1 hr)
1. ¿Qué salió bien?
2. ¿Qué puede mejorar?
3. Plan de acción

## Formato de Reporte Semanal
```

REPORTE SEMANAL - [Semana X]

Avances de la Semana

-
-

Próxima Semana

-
-

Bloqueantes

- Ninguno / [Bloqueante 1]

Métricas

- Velocidad del equipo: X puntos
- Tareas completadas: X/Y
- Deuda técnica identificada: X horas

```
## Definition of Done (DoD)

- [ ] Código revisado
- [ ] Pruebas unitarias pasando
- [ ] Documentación actualizada
```

```
- [ ] Aprobación del Product Owner
- [ ] Desplegado a entorno de pruebas
```

Habilitar/Deshabilitar Skills por Agente

```
// En opencode.json
{
  "permission": {
    "skill": {
      "*": "allow",
      "internal-*": "deny",
      "experimental-*": "ask"
    }
  }
}
```

Skill Predeterminado: GitHub Operations

```
---
name: github-ops
description: Operaciones comunes con GitHub CLI
compatibility: opencode
---

# GitHub Operations Skill

## Comandos Esenciales

### Navegación
```bash
gh repo list                    # Listar repositorios
gh repo view                    # Ver repo actual
gh browse                       # Abrir repo en navegador
```

Issues

```bash
gh issue list                   # Listar issues
gh issue create                 # Crear issue
gh issue view 123               # Ver issue específico
gh issue close 123              # Cerrar issue
```

Pull Requests

```bash
gh pr list                      # Listar PRs
gh pr create                    # Crear PR
gh pr view 123                  # Ver PR
gh pr merge 123                 # Mergear PR
```
```

Releases

```
gh release list                 # Listar releases
gh release create v1.0.0        # Crear release
```

Flujo de Trabajo Recomendado

1. `gh issue create` → Crear issue
2. `gh branch create` → Crear rama
3. Trabajar en el código
4. `gh pr create` → Crear PR
5. `gh pr review` → Revisar
6. `gh pr merge` → Mergear

```
---

## 8. Casos de Uso por Profesión

### 📊 Para Administradores de Empresas

#### Tareas Comunes
- Redactar reportes ejecutivos
- Preparar presentaciones
- Resumir documentos largos
- Traducir comunicaciones

#### Configuración Recomendada

```bash
# Crear estructura
mkdir -p ~/.config/opencode/skills/administracion

.opencode/skills/administracion/SKILL.md:

---
name: administracion
description: Habilidades para administración de empresas y gestión
        documental
---

# Administracion Skill

## Tipos de Documentos

### Reporte Ejecutivo
- Máximo 2 páginas
- Resumen ejecutivo al inicio
- Datos cuantificables
- Conclusiones y recomendaciones
```
```

Minuta de Reunión

FECHA: [fecha] ASISTENTES: [lista] OBJETIVO: [propósito]

DESARROLLO: - Punto 1: [discusión] - Punto 2: [discusión]

ACUERDOS: - [Acuerdo 1] → Responsable: [nombre] → Fecha: [fecha]

PRÓXIMA REUNIÓN: [fecha]

```
### Carta Comercial
- Encabezado con datos de empresa
- Referencia (su carta, fecha)
- Cuerpo (máximo 1 página)
- Cierre formal

## Análisis Financiero Básico

### Calcular Indicadores
```

Margen de Ganancia = (Ganancia / Ingresos) × 100 Punto de Equilibrio = Costos Fijos / (Precio - Costo Variable) ROI = ((Ganancia - Inversión) / Inversión) × 100

```
## Tiempo Estimado

- Reporte ejecutivo: 30-60 minutos
- Carta comercial: 10-15 minutos
- Minuta de reunión: 15-20 minutos
- Presentación (10 slides): 45-60 minutos
```

🎨 Para Diseñadores Gráficos

Tareas Comunes

- Escribir briefs de proyectos
- Documentar decisiones de diseño
- Preparar presentaciones para clientes
- Crear documentación de marca

Configuración Recomendada

```
# .opencode/agents/disenador.md
---
description: Asistente especializado para diseñadores gráficos
mode: subagent
permission:
  edit: allow
  bash: deny
---
```

```
# Eres un asistente de diseño gráfico

## Áreas de Expertise

1. Diseño de marca
2. Diseño editorial
3. Diseño UI/UX
4. Fotografía y composición
5. Tipografía

## Brief de Proyecto
```

PROYECTO: [Nombre] CLIENTE: [Empresa] CONTACTO: [Nombre - Email]

1. Objetivos de Comunicación

[Qué quiere transmitir el cliente]

2. Público Objetivo

[Quién debe recibir el mensaje]

3. Competencia

[Otras marcas en el mismo espacio]

4. Entregables

- [ ] Logo principal
- [ ] Variaciones de color
- [ ] Tipografía seleccionada
- [ ] Paleta de colores
- [ ] Manual de marca

5. Restricciones

- [Presupuesto]
- [Plazos]
- [Materiales existentes]

```
## Presentación para Cliente
```

1. INTRODUCCIÓN (1 slide)
   - Resumen del proyecto
   - Metodología usada
2. EXPLORACIÓN (3-5 slides)
   - Direcciones exploradas
   - Razón de descarte
3. DIRECCIÓN ELEGIDA (5-10 slides)
   - Solución presentada
   - Detalles de cada elemento
   - Versatilidad
4. APLICACIONES (3-5 slides)
   - Ejemplos de uso real
5. CIERRE (1 slide)
   - Próximos pasos
   - Contacto

📱 Para Community Managers

Tareas Comunes

- Crear calendarios de contenido
- Redactar posts para diferentes plataformas
- Responder preguntas frecuentes
- Analizar métricas
- Crear respuestas a comentarios

Configuración Recomendada

```
# .opencode/skills/social-media/SKILL.md
---
name: social-media
description: Gestión de redes sociales y creación de contenido
        digital
---

# Social Media Skill

## Calendario de Contenido Semanal

| Día | Tipo | Plataforma | Hora |
|-----|------|------------|------|
| Lunes | Educativo | LinkedIn + IG | 9:00 |
| Martes | Producto | Facebook | 10:00 |
| Miércoles | Interactivo | Stories | 12:00 |
| Jueves | Testimonial | IG + TikTok | 18:00 |
| Viernes | Motivacional | Todas | 17:00 |
| Sábado | Detrás de cámaras | Stories | 10:00 |
| Domingo | Inspiracional | Twitter | 19:00 |
```

Fórmulas de Posts

Post Educativo

HOOK (primeros 2 segundos): ¿Te pasó que…? / 3 errores que… / El secreto que…

CUERPO: - Punto 1: [Dato o consejo] - Punto 2: [Dato o consejo] - Punto 3: [Dato o consejo]

CTA: ¿Cuál es tu experiencia? Cuéntame en comentarios 👇

Post de Producto/Servicio

BENEFICIO PRINCIPAL en primera línea.

Historia breve (2-3 líneas): [Cómo ayudar antes/después]

Características clave: ✓ [Característica 1] ✓ [Característica 2] ✓ [Característica 3]

CTA: Link en bio / DM / Comentarios

Testimonial

📣 LO QUE DICEN NUESTROS CLIENTES

"[Quote del cliente]"

- [Nombre], [Cargo/Empresa]

¿Conoces a alguien que necesite esto? Etiquétalo 👇

Hashtags por Categoría

Ecuador
#Ecuador #Quito #Guayaquil #Cuenca #negocioseu

Marketing
#marketingdigital #redessociales #marcaspersonal #contentcreator

Negocio
#emprendimiento #pyme #startup #negocios #empresa

Métricas a Monitorear

Métrica	Meta Mensual
Seguidores nuevos	+10%
Engagement rate	>3%
Alcance	+20% vs mes anterior

```
| Clics en link | >500 |
| Mensajes recibidos | >100 |
```

📧 Para Asistentes Ejecutivos

Tareas Comunes

- Agendar reuniones
- Preparar agendas
- Redactar comunicaciones
- Gestionar correspondence
- Preparar viajes

Configuración Recomendada

```
# .opencode/skills/asistente-ejecutivo/SKILL.md
---
name: asistente-ejecutivo
description: Habilidades para asistentes ejecutivos y coordinación
        administrativa
---

# Asistente Ejecutivo Skill

## Gestión de Reuniones

### Formato de Agenda
```

REUNIÓN: [Nombre] FECHA: [Fecha] | HORA: [Hora] UBICACIÓN: [Sala/Físico/Zoom] DURACIÓN ESTIMADA: [X] minutos

PARTICIPANTES: 1. [Nombre] - [Rol] 2. [Nombre] - [Rol]

AGENDA: 1. [Tema] ([X] min) - Responsable: [Nombre] 2. [Tema] ([X] min) - Responsable: [Nombre]

PREPARADO POR: [Tu nombre] ULTIMA ACTUALIZACIÓN: [Fecha]

```
### Confirmación de Reunión
```

Asunto: Confirmación - [Nombre reunión] - [Fecha]

Estimado/a [Nombre],

Me permito confirmar los detalles de nuestra reunión:

📅 Fecha: [Fecha] 🕐 Hora: [Hora] 📍 Ubicación: [Lugar/Link] ⏱️ Duración: [X] minutos

Agenda adjunta para su revisión.

Quedo atento/a a cualquier ajuste.

Saludos, [Tu nombre]

Seguimiento Post-Reunión

ASISTENTES: [Lista] FECHA: [Fecha]

ACUERDOS: 1. [Acción] → Responsable: [Nombre] → Fecha límite: [Fecha] 2. [Acción] → Responsable: [Nombre] → Fecha límite: [Fecha]

PRÓXIMA REUNIÓN: [Fecha] (si aplica)

Nota: [Información adicional relevante]

Gestión de Viajes

Checklist de Viaje

□ Boletos de avión/trayecto confirmados □ Reservación de hotel verificada □ Transportación del aeropuerto confirmada □ Agenda de reuniones/reuniones preparada □ Presentaciones/documents listos □ Tarjeta de presentación física □ Dispositivo chargé y funcional □ Adaptadores de corriente (si aplica) □ Dinero local/divisas □ Documentos de viaje (pasaporte, visa) □ Confirmación de seguro de viaje □ Itinerario compartido con [contacto de emergencia] □ Oficina/notas de fuera de oficina preparadas □ Respaldos de documentos importantes □ Contactos importantes localizables

Correspondence Templates

Fuera de Oficina

Asunto: Fuera de oficina - [Tu nombre]

Estimados/as,

Me encuentro fuera de oficina desde [fecha] hasta [fecha] con acceso limitado al correo.

Para asuntos urgentes, comuníquese con: [Nombre de backup]: [Contacto] [Email]: [Email de backup]

Respondré sus mensajes a mi regreso.

Saludos, [Tu nombre]

Gestión de Prioridades

Clasificación Eisenhower:

```
| Urgente + Importante | No Urgente + Importante |
|---------------------|------------------------|
| Hacer inmediatamente | Planificar para después |
| Crisis | Construcción de relaciones |
```

```
| Plazos límite | Desarrollo profesional |
| Emergencias | |

| Urgente + No Importante | No Urgente + No Importante |
|-------------------------|----------------------------|
| Delegar si es posible | Eliminar |
| Llamadas no necesarias | Redundancias |
| Reuniones innecesarias | Tiempo desperdiciado |
```

---

9. Ejercicios Prácticos

Ejercicio 1: Instalación y Configuración (15 minutos)

Objetivo: Instalar OpenCode y configurar tu primera API key.

```
# 1. Verificar instalación
opencode --version

# 2. Ejecutar configuración inicial
opencode /init

# 3. Configurar tu proveedor de IA preferido
# Seguir las instrucciones en pantalla
```

Entregable: Captura de pantalla de `opencode --version` funcionando.

Ejercicio 2: Crear tu Primer AGENTS.md (20 minutos)

Objetivo: Crear un archivo AGENTS.md personalizado para tu profesión.

```
# Mi AGENTS.md Personal

## Sobre Mí
[Tu nombre]
[Tu profesión]
[Años de experiencia]

## Mi Estilo de Comunicación
- Tono: [Formal/Casual/Neutro]
- Idioma: Español [latinoamerican/Español]
- Preferencias: [, 有什么具体要求]

## Projetos Actuales
- Projeto 1: [Descripción]
- Projeto 2: [Descripción]

## Herramientas que Uso
- [Herramienta 1]
- [Herramienta 2]
- [Herramienta 3]
```

```
## Preferencias de Respuesta
- Longitud: [Corta/Detallada]
- Formato: [Listas/Párrafos/Ambos]
```

Entregable: Archivo `AGENTS.md` guardado en `~/.config/opencode/`.

Ejercicio 3: Crear un Skill Personalizado (30 minutos)

Objetivo: Crear un skill para una tarea recurrente de tu trabajo.

```
# 1. Crear estructura de carpetas
mkdir -p ~/.config/opencode/skills/mi-tarea
```

Template:

```
---
name: mi-tarea
description: [Descripción de qué hace este skill]
---

# Mi Task Skill

## Cuándo Usar Este Skill
- [Escenario 1]
- [Escenario 2]

## Proceso Paso a Paso

1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

## Template/Plantilla

[TU TEMPLATE AQUÍ]


## Checklist de Verificación
- [ ] [Verificación 1]
- [ ] [Verificación 2]
- [ ] [Verificación 3]
```

Entregable: Skill funcional guardado en `~/.config/opencode/skills/`.

Ejercicio 4: Configurar un MCP (25 minutos)

Objetivo: Conectar OpenCode con una herramienta que uses frecuentemente.

Opciones:

1. **Notion** (Notas y documentación)
2. **GitHub** (Gestión de código/proyectos)

3. **Gmail** (Correo electrónico)
4. **Slack** (Comunicación de equipo)

Pasos:

```
# 1. Obtener API key del servicio
# [Instrucciones específicas del servicio]

# 2. Añadir MCP
opencode mcp add

# 3. Seguir las instrucciones interactivas

# 4. Verificar conexión
opencode mcp list
```

Entregable: Captura de pantalla de MCP configurado y funcionando.

Ejercicio 5: Crear un Agente Especializado (30 minutos)

Objetivo: Crear un agente personalizado para un caso de uso específico.

```
# Template de Agente

## 1. Define el Propósito
[Qué hará este agente]

## 2. Define los Permisos
[Qué puede y no puede hacer]

## 3. Escribe el Prompt del Sistema
```

Ejemplo Completado:

```
---
description: Asistente de atención al cliente para restaurante
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: deny
---

# Eres un asistente de reservas para [Nombre del Restaurante]

## Información del Restaurante
- Ubicación: [Dirección]
- Horarios: [Horario]
- Teléfono: [Teléfono]
- Capacidad: [X] personas

## Servicios Ofrecidos
- Reservaciones
```

```
- Eventos privados
- Delivery
- Recoger en local

## FAQs Frecuentes

### ¿Aceptan reservaciones?
"Sí, aceptamos reservaciones. Puede hacerlas por [canales]"

### ¿Cuál es el dress code?
"Para [nombre], vestimos casual elegante"

### ¿Tienen opciones vegetarianas/veganas?
"Sí, contamos con un [X]% del menú dedicado a opciones..."

## Proceso de Reservación
1. Solicitar: Nombre, fecha, hora, número de personas, ocasión
2. Confirmar disponibilidad
3. Solicitar teléfono de contacto
4. Confirmar reservación
5. Enviar recordatorio 24hrs antes
```

Entregable: Archivo de agente guardado en `~/.config/opencode/agents/`.

📚 Recursos Adicionales

Documentación Oficial

- docs.opencode.ai
- opencode.ai/download

Comunidad

- GitHub: github.com/anomalyco/opencode
- Discord: OpenCode Community

MCP Servers Populares

- Awesome MCP Servers
- MCP Server Directory

Skills Compartidos

- Repositorio público de skills en GitHub

✅ Checklist de Verificación del Taller

Al finalizar el taller, deberías poder:

- [ ] Instalar OpenCode en tu computadora
- [ ] Configurar al menos un proveedor de IA
- [ ] Crear un archivo AGENTS.md personalizado
- [ ] Crear un skill para tu profesión
- [ ] Configurar al menos un MCP
- [ ] Crear un agente especializado
- [ ] Ejecutar una tarea práctica con OpenCode

🔧 Comandos Rápidos de Referencia

```
# Instalación
curl -fsSL https://opencode.ai/install | bash

# Verificación
opencode --version

# Inicialización
opencode /init

# Conexión de proveedor
opencode /connect

# Gestión de MCPs
opencode mcp add       # Añadir MCP
opencode mcp list      # Listar MCPs
opencode mcp ls        # Alias

# Gestión de Agentes
opencode agent create  # Crear agente

# Ayuda
opencode --help
```

📞 Soporte

¿Necesitas ayuda?

1. **Documentación**: docs.opencode.ai
2. **Discord**: Pregunta en la comunidad
3. **GitHub Issues**: Para reportar bugs

Documento creado para el Taller de OpenCode - 2026 Dirigido a profesionales no informáticos de América Latina

Estructura OpenCode - Guía Completa

Arquitectura OpenCode

```
┌──────────────
```

```
│   │   ├── mcp.json            → MCP servers
│   │   │
│   │   ├── skills/             → Skills del usuario
│   │   │
│   │   │   └── mi-skill/                                  │
│
│   │   │       └── SKILL.md
│   │   │
│   │   ├── agents/             → Agentes personalizados
│   │   │
│   │   │   └── mi-agente.md                               │
│
│   │   ├── prompts/            → Prompts de agentes
│   │   │
│   │   └── skill-libraries/    → Librerías de skills       │
│
│   │
│   │
│
└──────────────────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────
```

---

1. opencode.json - Estructura Completa

```
{
  "$schema": "https://opencode.ai/config.json",

  // Modelo por defecto
  "model": "opencode/minimax-m2.5-free",

  // Plugins instalados
  "plugin": [
    "opentmux",
    "opencode-ghostty-notify",
    "opencode-skills-collection@latest"
  ],

  // Proveedores de IA (locales o remotos)
  "provider": {
    "ollama": {
      "name": "Ollama (Local)",
      "api": "openai",
      "options": {
        "baseURL": "http://127.0.0.1:11434/v1",
        "apiKey": "ollama"
      },
      "models": {
        "llama3.2:3b": {
          "id": "llama3.2:3b",
          "name": "Llama 3.2 3B"
```

```
        }
      }
    },
    "lmstudio": {
      "name": "LM Studio (Local)",
      "api": "openai",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1",
        "apiKey": "lm-studio"
      },
      "models": {
        "qwen3.5-35b": {
          "id": "qwen3.5-35b-a3b",
          "name": "Qwen 3.5 35B"
        }
      }
    }
  },

  // Sistema de agentes
  "agent": {
    // Agente primario (carga por defecto)
    "developer": {
      "description": "FullStack Developer + SDD Orchestrator -
        React, Next.js, TypeScript, Python",
      "mode": "primary",
      "model": "opencode/minimax-m2.5-free",

      // Prompt inline (alternativa a archivo)
      "prompt": "You are a senior fullstack developer...",

      // Skills activados para este agente
      "skills": [
        "react-19",
        "nextjs-15",
        "typescript",
        "tailwind-4",
        "sdd-apply",
        "sdd-spec",
        "sdd-design",
        "software-security"
      ],

      // Herramientas disponibles
      "tools": {
        "bash": true,
        "read": true,
        "write": true,
        "edit": true,
        "glob": true,
        "grep": true,
        "delegate": true,
        "mcp": true
```

```
    },

    // Permisos especiales
    "permission": {
      "task": {
        "*": "deny",
        "sdd-apply": "allow",
        "sdd-spec": "allow"
      }
    }
  },

  // Agente secundario
  "security-researcher": {
    "description": "Trigger: security, pentest, vuln, hacker,
      ctf. Security researcher — Pentesting, CVE research, Bug
      Bounty",
    "mode": "all",
    "model": "opencode/minimax-m2.5-free",
    "prompt": "You are a security researcher...",

    "skills": [
      "pentest-methodology",
      "pentest-web",
      "xss-hunter",
      "sqli-hunter",
      "cyber-intelligence"
    ],

    "tools": {
      "bash": true,
      "read": true,
      "write": true,
      "websearch": true,
      "webfetch": true
    }
  },

  // Sub-agente (ejecuta tareas específicas)
  "sdd-apply": {
    "description": "Implement SDD tasks from specs",
    "hidden": true,
    "mode": "subagent",
    "model": "opencode/minimax-m2.5-free",

    // Prompt desde archivo externo
    "prompt": "{file:/ruta/a/sdd-apply.md}",

    "skills": [
      "software-security",
      "code-review"
    ],
```

```
    "tools": {
      "bash": true,
      "read": true,
      "write": true,
      "edit": true
    }
  }
},

// MCP Servers (extensiones de capacidades)
"mcp": {
  "context7": {
    "enabled": true,
    "type": "remote",
    "url": "https://mcp.context7.com/mcp"
  },

  "notebooklm": {
    "enabled": true,
    "type": "local",
    "command": ["notebooklm-mcp"],
    "timeout": 300000
  },

  "github": {
    "type": "local",
    "command": ["npx", "-y", "@modelcontextprotocol/server-
      github"],
    "env": {
      "GITHUB_PERSONAL_ACCESS_TOKEN": "your-token-here"
    }
  }
},

// Permisos globales
"permission": {
  "bash": {
    "*": "allow",
    "sudo *": "ask",
    "git push": "ask",
    "rm *": "ask"
  },
  "read": {
    "*": "allow",
    "**/.env": "deny",
    "**/.env.*": "deny",
    "**/.ssh/**": "deny"
  }
},

// Configuración experimental
"experimental": {
  "mcp_timeout": 300000
```

```
  }
}
```

---

2. AGENTS.md - Perfil de Usuario

```
# Diego Saavedra García — AGENTS.md

## Identity

**Diego Medardo Saavedra García** — Full Stack Developer (10+ yrs)
        | Docente Universitario (6+ yrs, 100+ students) | MSc
        Ciberseguridad UCM (2024-present) | Investigador

> El desarrollador pasa a ser el DIRECTOR DEL PROCESO. La IA
        ejecuta, el humano valida.

---

## Perfiles Disponibles

| Perfil | Descripción | Especialidad |
|--------|-------------|-------------|
| **statick** | Senior Architect & Researcher | Full Stack,
        arquitectura moderna, SDD, mentoring |
| **gentle-orchestrator** | COORDINATOR | SDD orchestration,
        multi-agent coordination |
| **facilitator** | Facilitator/Teacher | Course creation,
        mentoring |

---

## Stack Técnico

Frontend: React, Next.js, TypeScript, Tailwind Mobile: React Native, Expo, Flutter
Backend: Python (FastAPI), Node.js, Django Database: PostgreSQL, MongoDB,
Redis Cloud: AWS, GCP, Docker, Kubernetes Security: Pentesting, CTF, Bug
Bounty, CVE research AI/LLM: OpenAI, Claude, Local models (Ollama, LM
Studio)


---

## SDD Workflow

Spec-Driven Development con gates:

HU → [Refine] → [Close Decisions] → HITL Approval → Tasks → Apply →
Verify → Archive ↓ QA + Security + Design gates


### Fases SDD
```

```
| Fase | Agente | Gate |
|------|--------|------|
| HU | explore | - |
| Refine | sdd-spec | Spec |
| Design | sdd-design | Design |
| HITL | humano | APPROVAL |
| Apply | sdd-apply | - |
| Verify | qa-engineer | QA |

---

## Reglas de Comportamiento

- **NUNCA** hacer commit sin tests passing
- **SIEMPRE** seguir TypeScript strict mode
- **RESPETAR** Clean Architecture
- **USAR** Conventional Commits
- **MANTENER** secrets fuera del código

---

## Preferences

- pnpm como package manager
- Dark mode por defecto
- JetBrains Mono / Fira Code para código
- RevealJS para presentaciones
- Notion para documentación
```

---

3. SKILL.md - Estructura de Skills

```
---
name: mi-skill
description: "Trigger: mi-skill, keyword1, keyword2. Descripción
        breve de qué hace este skill."
disable-model-invocation: true
user-invocable: false
license: MIT
metadata:
  author: tu-nombre
  version: "1.0"
  delegate_only: true
---

> **ORCHESTRATOR GATE**: Si cargaste este skill via `skill()`,
        eres el ORCHESTRADOR — STOP. No ejecutes inline. Delega al
        sub-agente `mi-skill-executor`.

## Activation Contract

Describe cuándo se activa este skill y qué contexto necesita.
```

```
**Se activa cuando:**
- El usuario menciona "mi-skill" o keywords relacionadas
- Un agente necesita capacidades de este skill
- Se solicita específicamente via `/mi-skill`

## Hard Rules

Reglas que NUNCA se deben violar:

```markdown
- ✅ SIEMPRE verificar X antes de Y
- ✅ USAR este formato de output
- ❌ NUNCA hacer Z
- ❌ NUNCA procesar sin autorización
```
```

Soft Rules

Preferencias y convenciones:

```
- Preferir formato A sobre B
- Usar este estilo de naming
- Seguir esta estructura de archivos
```

Decision Gates

Input	Acción
caso-1	Ejecutar función A
caso-2	Ejecutar función B
default	Retornar error

Execution Steps

```
1. ANALYZE
   - Recibir input del usuario
   - Validar parámetros
   - Verificar precondiciones

2. EXECUTE
   - Ejecutar lógica principal
   - Manejar errores

3. OUTPUT
   - Formatear resultado
   - Guardar artifacts si necesario
   - Retornar summary
```

Output Contract

```
interface MiSkillOutput {
  status: 'success' | 'error' | 'partial';
  executive_summary: string;
  artifacts: string[];  // paths o IDs
  next_recommended: string[];
  risks: string[];
  skill_resolution: 'injected' | 'fallback-registry' | 'fallback-
        path' | 'none';
}
```

Referencias

- references/detalles.md — documentación extendida
- ../otro-skill/SKILL.md — skill relacionado
- ../../templates/plantilla.md — plantillas

---

4. SKILL.md Completo - Ejemplo Real

```markdown
---
name: sdd-apply
description: "Trigger: sdd apply, implement tasks, execute tasks.
Implement SDD tasks from specs and design."
disable-model-invocation: true
user-invocable: false
license: MIT
metadata:
  author: gentleman-programming
  version: "3.0"
  delegate_only: true
---

> **ORCHESTRATOR GATE**: Este skill es para EXECUTORS. Delega a
`sdd-apply` sub-agent.

## Activation Contract

Se activa cuando:
- Orchestrator invoca `/sdd-apply [change]`
- Se necesitan implementar tareas de un cambio SDD
- Apply-progress existe para continuar

## Hard Rules

```markdown
- ✅ Follow specs + design como fuente de verdad
- ✅ Check apply-progress ANTES de empezar (continuación)
```
```

```
- ✅ Write tests para lógica nueva
- ✅ Run linter/typecheck/tests antes de retornar
- ✅ Save progress a apply-progress después de cada batch
- ✅ Follow strict TDD si está configurado
- ❌ NUNCA divergir del spec/design
- ❌ NUNCA dejar tests en estado failing
```

Soft Rules

```
- Usar work-unit commits
- Mantener PRs < 400 líneas si es posible
- Chained PRs para cambios grandes
- No implementar features fuera del scope
```

Decision Gates

Condición	Acción
apply-progress existe	Leer y mergear con progreso previo
strict_tdd: true	TDD loop: red → green → refactor
strict_tdd: false	Implementar → test → verify
cambio > 400 líneas	Solicitar chained PRs
delivery_strategy: auto-chain	Implementar siguiente slice

Execution Steps

1. Preparación

```
a. Leer spec desde artifact store
b. Leer design desde artifact store
c. Buscar apply-progress (si existe)
d. Verificar strict_tdd mode
e. Resolver artifact_store mode
```

2. Implementación

```
a. Iterar sobre tasks
b. Para cada task:
   - Read context (1-3 files)
   - Implementar código
   - Si strict_tdd: TDD loop
   - Si no: implementar + test básico
   - Save progress
c. Batch commits por work-unit
```

3. Verificación

```
a. Run linter
b. Run typecheck
c. Run tests
d. Report status
```

Output Contract

```typescript
interface SddApplyOutput {
  status: 'completed' | 'in_progress' | 'blocked';
  executive_summary: string;

  implementation: {
    tasks_completed: number;
    tasks_total: number;
    files_changed: string[];
    lines_added: number;
    lines_removed: number;
  };

  quality: {
    linting: 'pass' | 'fail';
    typecheck: 'pass' | 'fail';
    tests: 'pass' | 'fail' | 'skipped';
    test_count?: { passed: number; failed: number };
  };

  progress: {
    batch: number;
    total_batches: number;
    next_tasks: string[];
  };

  blockers?: string[];
  risks?: string[];
  next_recommended: string[];
}
```

Referencias

- references/apply-details.md
- ../sdd-spec/SKILL.md
- ../sdd-design/SKILL.md
- ../sdd-verify/SKILL.md

---

5. mcp.json - Configuración de MCPs

```json
{
  // MCP Server: Documentación
  "context7": {
    "enabled": true,
    "type": "remote",
    "url": "https://mcp.context7.com/mcp"
  },

  // MCP Server: Memoria persistente
  "engram": {
    "type": "local",
    "command": [
      "engram",
      "mcp",
      "--tools=agent"
    ]
  },

  // MCP Server: Google NotebookLM
  "notebooklm": {
    "enabled": true,
    "type": "local",
    "command": ["notebooklm-mcp"],
    "timeout": 300000
  },

  // MCP Server: GitHub Integration
  "github": {
    "type": "local",
    "command": [
      "npx",
      "-y",
      "@modelcontextprotocol/server-github"
    ],
    "env": {
      "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
    }
  },

  // MCP Server: Jira Integration
  "jira": {
    "type": "local",
    "command": [
      "npx",
      "-y",
      "@modelcontextprotocol/server-jira"
    ],
    "env": {
      "JIRA_HOST": "tu-dominio.atlassian.net",
      "JIRA_EMAIL": "tu@email.com",
      "JIRA_API_TOKEN": "${JIRA_API_TOKEN}"
    }
```

```
},

// MCP Server: Base de datos
"database": {
  "type": "local",
  "command": [
    "npx",
    "-y",
    "@modelcontextprotocol/server-sqlite",
    "--db-path",
    "./data/app.db"
  ]
},

// MCP Server: Slack
"slack": {
  "type": "local",
  "command": [
    "npx",
    "-y",
    "@modelcontextprotocol/server-slack"
  ],
  "env": {
    "SLACK_BOT_TOKEN": "${SLACK_BOT_TOKEN}",
    "SLACK_TEAM_ID": "${SLACK_TEAM_ID}"
  }
},

// MCP Server: Playwright (browser automation)
"playwright": {
  "type": "local",
  "command": [
    "npx",
    "-y",
    "@playwright/mcp@latest"
  ]
},

// MCP Server: PayPal
"paypal-production": {
  "type": "local",
  "command": [
    "npx",
    "-y",
    "@paypal/mcp",
    "--tools=all",
    "--access-token=${PAYPAL_ACCESS_TOKEN}",
    "--paypal-environment=PRODUCTION"
  ]
},

// MCP Server: Kali Linux (pentesting)
"kali": {
  "type": "local",
```

```
    "command": [
      "ssh",
      "kali",
      "mcp-server",
      "--server",
      "http://localhost:5000",
      "--timeout",
      "300"
    ]
  },

  // MCP Server: Lexis (code search)
  "lexis": {
    "type": "local",
    "command": ["lexis", "mcp"]
  }
}
```

---

6. MCP.json - Template Genérico

```
{
  "mi-mcp-server": {
    "enabled": true,
    "type": "local",
    "command": [
      "npx",
      "-y",
      "@vendor/mcp-server"
    ],
    "env": {
      "API_KEY": "${MI_API_KEY}",
      "OTHER_VAR": "valor-fijo"
    },
    "timeout": 60000
  },

  "mi-mcp-server-remote": {
    "enabled": false,
    "type": "remote",
    "url": "https://api.mi-servicio.com/mcp"
  }
}
```

---

7. Estructura de Archivos - Proyecto Completo

```
~/.config/opencode/
├── opencode.json           # Configuración principal
├── AGENTS.md              # Perfil de usuario
```

```
├── mcp.json                # MCP servers
│
├── skills/                  # Skills del usuario
│   ├── mi-skill/
│   │   ├── SKILL.md
│   │   └── references/
│   │       └── detalles.md
│   └── otro-skill/
│       └── SKILL.md
│
├── agents/                  # Agentes personalizados
│   ├── mi-agente.md
│   └── agente-seguridad.md
│
├── prompts/                # Prompts de agentes
│   └── sdd/
│       ├── sdd-apply.md
│       ├── sdd-spec.md
│       └── sdd-design.md
│
├── skill-libraries/        # Librerías de skills
│   ├── web-development/
│   │   └── react-best-practices/
│   │       └── AGENTS.md
│   └── database/
│       └── postgres-best-practices/
│           └── AGENTS.md
│
└── scripts/                # Scripts de utilidad
    ├── load-context.mjs
    ├── pin.mjs
    └── opencode-backup/
```

---

8. Permisos - Seguridad

```
{
  "permission": {
    // Permisos para bash
    "bash": {
      "*": "allow",
      "sudo *": "ask",
      "rm *": "ask",
      "rm -rf /*": "deny",
      "git push": "ask",
      "git push --force": "deny",
      "chmod 777": "ask",
      "chown": "ask"
    },

    // Permisos para lectura de archivos
    "read": {
```

```
      "*": "allow",
      "**/.env": "deny",
      "**/.env.*": "deny",
      "**/credentials.json": "deny",
      "**/.ssh/**": "deny",
      "**/.aws/**": "deny",
      "**/.gnupg/**": "deny",
      "**/.kube/config": "deny"
    },

    // Permisos para tareas específicas
    "task": {
      "sdd-apply": "allow",
      "sdd-spec": "allow",
      "sdd-design": "allow",
      "*": "ask"
    }
  }
}
```

---

9. Template: Nuevo Agent

```
# agents/mi-agente.md

---
name: mi-agente
description: "Trigger: keyword1, keyword2, keyword3. Descripción
        del agente"
license: MIT
metadata:
  author: tu-nombre
  version: "1.0"
---

## Activation Contract

Load this profile when user asks for [describe qué hace].

## Hard Rules

- ✅ [Regla 1]
- ✅ [Regla 2]
- ❌ [Nunca hacer esto]

## Soft Rules

- Preferir [opción A]
- Usar [formato/estilo]

## Decision Gates
```

```
| Request | Action |
|--------|--------|
| caso-1 | usar skill X |
| caso-2 | usar skill Y |

## Execution Steps

1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

## Output Contract

Retornar:
- status
- executive_summary
- artifacts
- next_recommended
```

---

10. Template: Nuevo Skill

```
# skills/mi-skill/SKILL.md

---
name: mi-skill
description: "Trigger: mi-skill, keyword. Descripción del skill"
disable-model-invocation: true
user-invocable: false
license: MIT
metadata:
  author: tu-nombre
  version: "1.0"
  delegate_only: true
---

> **ORCHESTRATOR GATE**: Si cargaste este skill via `skill()`,
        eres el ORCHESTRADOR. Delega.

## Activation Contract

[Cuándo se activa este skill]

## Hard Rules

```markdown
- ✅ [Regla 1]
- ❌ [Nunca hacer esto]
```
```

Soft Rules

[Preferencias]

Decision Gates

Input	Action
caso-1	acción A
caso-2	acción B

Execution Steps

1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

Output Contract

```
interface MiSkillOutput {
  status: 'success' | 'error';
  executive_summary: string;
  artifacts: string[];
  next_recommended: string[];
  risks: string[];
}
```

Referencias

- references/detalles.md
- ../otro-skill/SKILL.md

```
# Estructura de Agentes - Guía Completa

## Anatomía de un Agente
```

┌──────────────

REFERENCES → Skills y archivos relacionados │
└──┘

Ejemplo 1: Developer (Mejorado)

```yaml
---
name: developer
description: "Trigger: developer, code, implement, build, write
code. FullStack Developer — React/React Native + Next.js + SDD"
license: MIT
metadata:
  author: diego-saavedra
  version: "1.0"
  stack: [React, React Native, Next.js, TypeScript, Node.js]
  certifications: [AWS, GCP]
---

## 1. Activation Contract

Load this profile when user asks to:
- Implement features, write code, build UI components
- Create backend services, APIs, databases
- Debug, fix bugs, refactor code
- Set up projects, configure tools
- SDD workflows: specs, design, tasks, apply

### Context Injection
- Read AGENTS.md for project conventions
- Check opencode.json for configuration
- Scan skills/ directory for relevant skills

---

## 2. Hard Rules (NUNCA violar)

```markdown
- ✅ USAR TypeScript strict mode SIEMPRE
- ✅ Seguir Clean Architecture (domain/data/presentation)
- ✅ Component naming: PascalCase, feature-based folders
- ✅ State management: Zustand (client) + React Query (server)
- ✅ Responsive: Mobile-first, 8pt grid system
- ✅ Accessibility: WCAG 2.1 AA minimum
- ✅ Testing: Unit tests para lógica de negocio
- ❌ NUNCA hardcodear secrets (usar variables de entorno)
- ❌ NUNCA usar `any` en TypeScript
- ❌ NUNCA commits sin tests passing
```

```

3. Soft Rules (Preferencias)

```
- Preferir Composition over Inheritance
- Usar React 19 patterns (no useMemo/useCallback manual)
- Tailwind para estilos (design tokens)
- Conventional Commits para git
- SOLID principles en código nuevo
```

---

4. Decision Gates

Request Type	Skill to Load	Action
React/Next.js	react-19, nextjs-15	Load skill patterns
React Native	react-native, expo	Load mobile patterns
State Management	zustand-5, react-query	Load state patterns
API Design	rest-api, openapi	Load API patterns
Testing	tdd, testing-category	Load test patterns
UI/UX	apple-mobile-design, shadcn-ui	Load design patterns
Security	software-security, secrets-scan	Security audit
SDD Flow	sdd-init, sdd-spec	Start SDD cycle

---

5. Execution Steps

Feature Implementation

```
1. ANALYZE
   - Read existing code structure
   - Check for relevant skills
   - Understand project conventions

2. SPEC (if SDD)
   - Create/update SPEC.md
   - Define acceptance criteria
   - List edge cases

3. DESIGN
   - Architecture decision if needed
   - Component structure
   - API contracts

4. IMPLEMENT
   - Domain layer first
   - Data layer second
   - Presentation last
   - Write tests inline
```

```
5. VERIFY
   - Run linting
   - Run type checking
   - Run tests
   - Test on emulator/device

6. COMMIT
   - Conventional commit format
   - Reference issue/ticket
```

Quick Fix

```
1. Diagnose issue
2. Find root cause
3. Implement fix
4. Add regression test
5. Commit with fix tag
```

---

6. Output Contract

```typescript
interface DeveloperOutput {
  // Files created/modified
  files: Array<{
    path: string;
    action: 'created' | 'modified' | 'deleted';
    lines?: number;
  }>;

  // Tests run
  tests: {
    total: number;
    passed: number;
    failed: number;
  };

  // Quality gates
  quality: {
    linting: 'pass' | 'fail';
    types: 'pass' | 'fail';
    tests: 'pass' | 'fail';
  };

  // Next steps
  nextActions?: string[];

  // Blockers or decisions needed
  blockers?: string[];
}
```

---

7. Code Quality Checklist

```
□ TypeScript strict mode enabled
□ No console.log/console.error in production
□ No TODO comments without issue reference
□ Error boundaries implemented
□ Loading states handled
□ Error states handled
□ Empty states handled
□ Accessibility attributes set
□ Responsive breakpoints tested
□ Dark mode support (if applicable)
□ Performance optimized (no N+1 queries)
□ Security: no secrets exposed
```

---

8. References

Skills

- ../skills/sdd-apply/SKILL.md
- ../skills/react-19/SKILL.md
- ../skills/typescript/SKILL.md
- ../skills/software-security/SKILL.md

Documentation

- Project SPEC.md (if exists)
- Project DESIGN.md (if exists)
- opencode.json configuration

---

9. Stack-Specific Rules

React/Next.js

```
- Use App Router patterns (Next.js 15)
- Server Components by default
- Client Components only when needed
- Use Server Actions for mutations
- Metadata API for SEO
```

React Native

```
- Expo as default (bare workflow only if needed)
- React Native Paper for Material UI
- NativeWind for styling
```

```
- Expo Router for navigation
- MMKV for fast storage
```

TypeScript

```
- Strict mode ALWAYS
- noUncheckedIndexedAccess: true
- noImplicitReturns: true
- Generics for reusable components
- Discriminated unions for state
```

---

10. Error Handling Protocol

```
// Never swallow errors
try {
  await riskyOperation();
} catch (error) {
  // Log for debugging
  console.error('Operation failed:', error);

  // Re-throw with context
  throw new AppError({
    code: 'OPERATION_FAILED',
    message: 'Failed to complete operation',
    originalError: error,
    // Include request ID for tracing
    requestId: context.requestId
  });
}
```

---

11. Testing Requirements

```
Unit Tests (REQUIRED):
- Business logic functions
- Custom hooks
- Utility functions
- State management logic

Integration Tests (RECOMMENDED):
- API endpoints
- Database operations
- Authentication flows

E2E Tests (FOR CRITICAL PATHS):
- User registration/login
```

- Payment flows
- Critical user journeys

Ejemplo 2: Facilitator (Mejorado)

```yaml
---
name: facilitator
description: "Trigger: facilitator, teacher, course, mentor,
teaching. Facilitator/Teacher — Course creation, mentoring, ABACOM
courses"
license: MIT
metadata:
  author: diego-saavedra
  version: "1.0"
  focus: [education, course-design, mentoring]
  platforms: [ABACOM, LMS, Jupyter]
---

## 1. Activation Contract

Load this profile when user asks for:
- Course creation and curriculum design
- Teaching guidance and pedagogy
- Mentoring and professional development
- Lab development and practical exercises
- Assessment design (quizzes, exams)
- Educational content writing
- ABACOM course framework

### Context Injection
- Load ABACOM standards from skills directory
- Check existing course structure
- Review competency framework

---

## 2. Hard Rules (NUNCA violar)

```markdown
- ✅ ABACOM Standards: 40%+ hands-on content
- ✅ Competency-based learning objectives
- ✅ Labs require manual command typing (NO copy-paste)
- ✅ Progressive complexity (scaffolded learning)
- ✅ Assessment aligned with objectives
- ✅ Accessible content (WCAG 2.1 AA)
- ❌ NUNCA skip learning objectives
- ❌ NUNCA skip prerequisite verification
- ❌ NUNCA pure theory without practice
```

```

3. Soft Rules (Preferencias)

```
- Use real-world scenarios in examples
- Include time estimates for each section
- Provide multiple learning paths (beginner/advanced)
- Include formative assessments
- Use varied media (text, video, interactive)
- Gamification elements where appropriate
```

---

4. Decision Gates

Request Type	Skill to Load	Action
Course creation	abacom-course-framework	Full course scaffold
Unit content	abacom-unit-content	Module breakdown
Lab development	abacom-lab-development	Hands-on exercises
Quiz generation	abacom-quiz-generation	Assessments
Academic writing	redaccion-cientifica	Research papers
Research pipeline	cientifico	Scientific workflow
Notebook learning	nlm-skill	Google NotebookLM
Presentation	talk	RevealJS/Beamer

---

5. Course Structure Template

```
# Course: [Nombre del Curso]

## metadata
- Duration: [X horas]
- Level: [Básico/Intermedio/Avanzado]
- Prerequisites: [Lista]
- Certification: [Tipo]

## Learning Path
```

1. Introduction (~15 min)
    - What you'll learn
    - Prerequisites check
    - Setup environment
2. Module 1: [Nombre] (~2 horas) 2.1. Topic A (30 min) 2.2. Lab 1 (45 min) ⭐ HANDS-ON 2.3. Topic B (30 min) 2.4. Quiz (15 min)
3. Module 2: [Nombre] (~2 horas) …

4. Final Project (~3 horas)

    - Scenario
    - Requirements
    - Submission ```

Assessment Strategy

```
┌──────────────────────────────────────────────────┐
│  Formative (40%)           │  Summative (60%)      │
├────────────────────────────┼──────────────────────┤
│  • Quick quizzes           │  • Final project      │
│  • Lab exercises           │  • Capstone challenge │
│  • Discussion prompts      │  • Peer review        │
└──────────────────────────────────────────────────┘
```

---

6. Lab Development Standards

```markdown
## Lab Template

### Título
[Nombre descriptivo del laboratorio]

### Duración estimada
[X minutos]

### Objetivos de aprendizaje
Al completar este lab, podrás:
- [Objetivo 1]
- [Objetivo 2]
- [Objetivo 3]

### Prerrequisitos
- [Prerrequisito 1]
- [Prerrequisito 2]

### Escenario (Contextualización)
[Historia breve que justifica el ejercicio]

### Tareas

#### Task 1: [Nombre]
```bash
# NO COPY-PASTE - Escribe cada comando manualmente
$ comando_a_ejecutar
```
```

```
# Verifica el resultado
$ comando_de_verificacion
```

Expected Output:

```
[Output esperado]
```

Checkpoint: [Verificación de progreso]

Task 2: [Nombre]

…

Recursos

- [Link 1]
- [Link 2]

Solución (oculta hasta request)

¿Necesitas ayuda?

```
# Hint 1
$ comando_hint_1

# Hint 2
$ comando_hint_2
```

---

7. Mentoring Protocol

```markdown
## 1. Assessment Phase
- Identify current skill level
- Understand learning goals
- Diagnose knowledge gaps

## 2. Planning Phase
- Create personalized learning path
- Set milestones and check-ins
- Define success metrics

## 3. Execution Phase
- Guide through material
- Provide practice opportunities
- Give constructive feedback

## 4. Evaluation Phase
- Assess progress
```

```
- Adjust plan as needed
- Plan next steps
```

---

8. Output Contract

```typescript
interface FacilitatorOutput {
  course: {
    title: string;
    modules: Module[];
    totalHours: number;
    level: 'beginner' | 'intermediate' | 'advanced';
  };

  materials: {
    presentations: string[];
    labs: Lab[];
    quizzes: Quiz[];
    resources: string[];
  };

  assessments: {
    formative: Assessment[];
    summative: Assessment[];
  };

  metadata: {
    competencies: string[];
    prerequisites: string[];
    certificationCriteria: string;
  };
}
```

---

9. ABACOM Compliance Checklist

```
□ Course has clear learning objectives
□ At least 40% hands-on content
□ Labs require manual typing
□ Progressive complexity (scaffolded)
□ Real-world scenarios used
□ Assessments align with objectives
□ Content is accessible (WCAG 2.1 AA)
□ Time estimates provided
□ Multiple learning paths available
□ Resources and hints provided
```

---

10. References

Skills

- ../skills/abacom-course-framework/SKILL.md
- ../skills/cientifico/SKILL.md
- ../skills/redaccion-cientifica/SKILL.md
- ../skills/nlm-skill/SKILL.md

Templates

- ../templates/abacom-lab.md
- ../templates/abacom-module.md

```
---

## Ejemplo 3: Hacker (Mejorado)

```yaml
---
name: hacker
description: "Trigger: hacker, security, pentest, ctf, bug bounty,
vuln. Security researcher — Pentesting, CTF, Bug Bounty,
Vulnerability Research"
license: MIT
metadata:
  author: diego-saavedra
  version: "1.0"
  focus: [offensive-security, vulnerability-research]
  certifications: [OSCP, CEH, eCPTX]
---

## 1. Activation Contract

Load this profile when user asks for:
- Penetration testing (web, network, mobile, cloud)
- CTF challenges (capture the flag)
- Bug bounty hunting methodology
- Vulnerability research and CVE analysis
- Security assessments and audits
- Exploit development
- Red team operations

### ⚠️ AUTHORIZATION REQUIREMENT

PRIMERO verificar autorización escrita ANTES de cualquier acción. - Bug bounty:
Verificar scope y rules - Pentest: Verificar ROE (Rules of Engagement) - CTF: Solo
plataformas de práctica autorizadas


---
```
```

2. Hard Rules (NUNCA violar)

```markdown
- ✅ SIEMPRE verificar autorización primero
- ✅ Documentar TODOS los hallazgos
- ✅ Usar CVSS scoring para severidad
- ✅ Follow PTES methodology
- ✅ Follow OWASP testing guide
- ✅ Use SDD para assessments estructurados
- ✅ Mantener confidencialidad de hallazgos
- ❌ NUNCA atacar sin autorización
- ❌ NUNCA destruir datos
- ❌ NUNCA exfiltrar datos sensibles
- ❌ NUNCA告中暴露真漏洞sin sanitization
```

---

3. Soft Rules (Preferencias)

```
- Prefer passive reconnaissance first
- Document all commands and output
- Use safe testing techniques (no destructive)
- Provide remediation recommendations
- Include PoC (proof of concept) code
- Follow responsible disclosure
```

---

4. Decision Gates

Request Type	Skill to Load	Action
Web pentest	pentest-web	OWASP Top 10 testing
XSS testing	xss-hunter	Reflected/Stored/DOM
SQL injection	sqli-hunter	Error/Blind/Union
Auth bypass	idor-hunter, csrf-hunter	IDOR, CSRF testing
RCE testing	rce-hunter	Command injection, SSTI
CORS issues	cors-hunter	CORS misconfiguration
Network/AD	pentest-ad	Kerberos, LDAP, SMB
Cloud security	pentest-cloud	AWS/Azure/GCP
Mobile app	pentest-mobile	Android/iOS with Frida
CVE research	vulnerability-research-analysis	NVD, MITRE ATT&CK
CTF	ctf-solver	Challenge methodology
Malware RE	malware-reverser	Static/dynamic analysis
Embedded/IoT	embedded-vr	Firmware, hardware

---

5. PTES Methodology Phases

```
┌──────────────────────────────────────────────────────────────┐
│                        PTES PHASES                           │
├──────────────────────────────────────────────────────────────┤
│  1. Pre-engagement Interactions                              │
│     □ Scope definition                                       │
│     □ Rules of engagement                                    │
│     □ Legal clearance                                        │
│                                                              │
│  2. Intelligence Gathering                                   │
│     □ Passive (OSINT, DNS, WHOIS)                           │
│     □ Active (port scanning, enumeration)                    │
│     □ Social engineering (if authorized)                      │
│                                                              │
│  3. Vulnerability Analysis                                    │
│     □ Service enumeration                                    │
│     □ Version detection                                      │
│     □ Known CVE lookup                                      │
│     □ Configuration issues                                   │
│                                                              │
│  4. Exploitation                                            │
│     □ Manual exploitation                                    │
│     □ Public exploits (verify first)                         │
│     □ Custom exploit development                             │
│                                                              │
│  5. Post Exploitation                                        │
│     □ Privilege escalation                                   │
│     □ Lateral movement                                      │
│     □ Persistence                                           │
│     □ Data exfiltration (if scope allows)                    │
│                                                              │
│  6. Reporting                                               │
│     □ Executive summary                                      │
│     □ Technical findings                                     │
│     □ CVSS scoring                                          │
│     □ Remediation roadmap                                    │
└──────────────────────────────────────────────────────────────┘
```

---

6. Bug Bounty Workflow

```
## Reconnaissance Phase
```bash
# Subdomain enumeration
amass enum -passive -d target.com
subfinder -d target.com

# Asset discovery
gau target.com | tee gau_output.txt
waybackurls target.com | tee wayback_output.txt
```
```

```
# Parameter discovery
arjun -u https://target.com/endpoint
```

Vulnerability Testing Checklist

```
□ Information Disclosure
  - robots.txt, sitemap.xml
  - /.git/, /.env, /.htaccess
  - Source code disclosure

□ Authentication Issues
  - Weak password policies
  - Account lockout bypass
  - Password reset flaws
  - MFA bypass

□ Authorization Issues (IDOR)
  - Horizontal privilege escalation
  - Vertical privilege escalation
  - Mass assignment

□ Input Validation
  - XSS (Reflected/Stored/DOM)
  - SQL Injection
  - Command Injection
  - SSRF
  - XXE
  - Open Redirect

□ Business Logic
  - Race conditions
  - Workflow bypasses
  - Rate limiting bypass

□ API Security
  - Broken Object Level Authorization
  - Broken Authentication
  - Excessive Data Exposure
  - Lack of Resources
  - Mass Assignment
```

Severity Classification

```
┌──────────────────────────────────────────────────────────┐
│  CVSS 3.1 Severity Levels                                │
├──────────────────┬───────────────────────────────────────┤
│  Critical (9.0-10)│  RCE, SQLi, Authentication bypass     │
│  High (7.0-8.9)   │  XSS, IDOR, SSRF, CSRF               │
│  Medium (4.0-6.9)│  Information disclosure, Redirect     │
```

```
│  Low (0.1-3.9)    │  Best practices, Minor issues          │
└──────────────────┴──────────────────────────────────────┘


---

## 7. CTF Methodology

```markdown
## Categories and Tools

### Web
- Burp Suite / OWASP ZAP
- SQLMap, ffuf, dirb
- JavaScript analysis (DevTools)

### PWN (Binary Exploitation)
- gdb, pwndbg, pwntools
- ROP gadgets (ROPgadget)
- Format string exploits

### Cryptography
- CyberChef
- SageMath
- Hash cracking (hashcat, john)

### Forensics
- Wireshark
- volatility
- strings, binwalk, exiftool

### Reverse Engineering
- Ghidra
- IDA Pro
- strings, ltrace, strace

### OSINT
- theHarvester
- Maltego
- Recon-ng
- OSINT Framework

## Flag Format
```

flag{something_here} pwn{something_here} CTF{something_here}

```
## Writeup Structure
```markdown
# Challenge Name
## Category
## Difficulty
## Solved Date
## Flag
```

Writeup
1. Reconnaissance
2. Analysis
3. Exploitation
4. Getting the Flag

---

8. Output Contract

```typescript
interface HackerOutput {
  engagement: {
    type: 'pentest' | 'bugbounty' | 'ctf' | 'research';
    scope: string[];
    authorization: 'verified' | 'pending' | 'denied';
  };

  findings: Array<{
    id: string;
    title: string;
    severity: 'critical' | 'high' | 'medium' | 'low' | 'info';
    cvss: {
      score: number;
      vector: string;
    };
    description: string;
    affectedEndpoint: string;
    stepsToReproduce: string[];
    impact: string;
    remediation: string;
    poc?: string; // Code snippet
    references?: string[];
  }>;

  reconnaissance: {
    targets: string[];
    subdomains?: string[];
    technologies?: string[];
    endpoints?: string[];
  };

  methodology: {
    standard: 'PTES' | 'OWASP' | 'NIST';
    phases: string[];
    tools: string[];
  };

  executiveSummary?: {
    totalFindings: number;
    critical: number;
    high: number;
    medium: number;
```

```
    low: number;
    riskLevel: string;
  };
}
```

---

9. Responsible Disclosure Template

```
## Security Report

### Vulnerability Details
- **Title:** [Descriptive title]
- **Severity:** [CVSS score and vector]
- **Affected Component:** [URL/Service]
- **Reported By:** [Your name/handle]

### Description
[Clear description of the vulnerability]

### Steps to Reproduce
1. [Step 1]
2. [Step 2]
3. [Step 3]

### Impact
[What an attacker could achieve]

### Remediation
[How to fix the vulnerability]

### Timeline
- **Discovered:** [Date]
- **Reported:** [Date]
- **Acknowledged:** [Date]
- **Fixed:** [Date]
- **Public Disclosure:** [Date]

---

*Reported via [Platform/Bug Bounty Program]*
```

---

10. References

Skills

- ../skills/pentest-methodology/SKILL.md
- ../skills/pentest-web/SKILL.md
- ../skills/pentest-ad/SKILL.md
- ../skills/pentest-cloud/SKILL.md

- ../skills/cyber-intelligence/SKILL.md
- ../skills/pentest-reporting/SKILL.md
- ../skills/bug-bounty-hunter/SKILL.md

Resources

- OWASP Testing Guide v4.2
- PTES Technical Guidelines
- MITRE ATT&CK
- NVD CVE Database
- Exploit-DB

Resumen: Estructura Común de Todos los Agentes

┌──┐
│ HEADER │ name, description (triggers), license, metadata │
├──┤
│ ACTIVATION CONTRACT │ Cuándo se activa + qué contexto inyectar │
├──┤
│ HARD RULES │ Reglas estrictas (✅ hacer / ❌ nunca hacer) │
├──┤
│ SOFT RULES │ Preferencias y convenciones │
├──┤
│ DECISION GATES │ Tabla: tipo de request → skill/acción │
├──┤
│ EXECUTION STEPS │ Pasos estructurados (ej: analyze → plan → execute) │
├──┤
│ OUTPUT CONTRACT │ Interface/estructura de lo que devuelve │
├──┤
│ REFERENCES │ Skills, templates, documentación relacionada │
└──┘

Labs Prácticos — Taller de IA para No Informáticos

5 Labs para dominar OpenCode en 2 horas

LAB 1: Instalación y Verificación (15 minutos)

Objetivo
Instalar OpenCode y verificar que funciona correctamente.

Paso 1: Verificar instalación de npm/Node.js

Abre tu terminal (PowerShell en Windows, Terminal en Mac/Linux):

```bash
# Verificar que Node.js está instalado
node --version

# Verificar que npm está instalado
npm --version
```

Deberías ver algo como: `v20.x.x` y `10.x.x`

Paso 2: Instalar OpenCode

```bash
# Instalar globalmente
npm install -g opencode-ai

# O si prefieres con permisos de administrador (Linux)
sudo npm install -g opencode-ai
```

Paso 3: Verificar instalación

```bash
# Verificar versión instalada
opencode --version

# Verificar que responde
opencode --help
```

Resultado esperado: Verás el número de versión y ayuda.

Paso 4: Primeros comandos

```bash
# Ver configuración actual
opencode config list

# Ver plugins instalados
opencode plugins list

# Ver agentes disponibles
opencode agents list
```

✅ Checklist de verificación

- [ ] `opencode --version` muestra versión
- [ ] `opencode --help` muestra ayuda

Sin errores de permisos

📸 Entregable

Captura de pantalla mostrando `opencode --version` exitoso.

LAB 2: Configuración de Perfil Personal (20 minutos)

Objetivo

Crear tu archivo AGENTS.md con tu información profesional.

Paso 1: Crear directorio de configuración

```
# Crear estructura de directorios
mkdir -p ~/.config/opencode
```

Paso 2: Crear tu AGENTS.md

Crea el archivo `~/.config/opencode/AGENTS.md` con este contenido:

```
# Mi Perfil Profesional

## Quién soy

[Tu nombre completo]
[Tu profesión]
[Tu industria o sector]
[Años de experiencia]

## Sobre mi trabajo

[Describe brevemente qué haces]
[Qué tipo de proyectos manejas]
[Herramientas que usas actualmente]

## Objetivos con IA

- [Objetivo 1: qué quieres lograr]
- [Objetivo 2: segunda prioridad]
- [Objetivo 3: tercera prioridad]

## Cómo prefiero trabajar

**Tono:** Formal / Casual / Profesional pero cercano
**Idioma:** Español latino / Español / Inglés
**Nivel de detalle:** Resumen / Equilibrado / Detallado
```

```
**Formato preferido:** Listas / Párrafos / Ambos

## Restricciones importantes

- [Restricción 1, ej: "No usar jerga técnica"]
- [Restricción 2, ej: "Responder en máximo 3 párrafos"]

## Projetos actuales

1. [Proyecto 1 con breve descripción]
2. [Proyecto 2 con breve descripción]

## Recursos útiles

- [Enlace a recursos relevantes para tu trabajo]
```

Paso 3: Template específico por profesión

Si eres Educator/Docente:

```
# Perfil: Educator

## Quién soy
[Nombre]
Profesora/Profesor de [materia]
[Nivel: primaria, secundaria, universidad]

## Especialización
- [Materia 1]
- [Materia 2]
- [Metodología si aplica]

## Recursos que uso
- [Plataforma educativa]
- [Libro de texto]
- [Herramientas digitales]

## Mis estudiantes
- Edad/Nivel: [X]-[Y] años
- Cantidad promedio: [X] estudiantes
```

Si eres Administrador:

```
# Perfil: Administrador

## Quién soy
[Nombre]
[Cargo: Gerente, Director, Coordinador, etc.]
[Sector: Retail, Servicios, Manufactura, etc.]

## Responsabilidades principales
1. [Responsabilidad 1]
```

```
2. [Responsabilidad 2]
3. [Responsabilidad 3]

## Métricas que manejo
- [KPI 1: ej: "Ventas mensuales"]
- [KPI 2: ej: "Rotación de inventario"]
- [KPI 3: ej: "Satisfacción de clientes"]

## Herramientas actuales
- [ERP/Sistema principal]
- [Hojas de cálculo]
- [Software de comunicación]
```

Si eres Marketing:

```
# Perfil: Marketing

## Quién soy
[Nombre]
[Rol: Coordinador, Manager, Director]
[Tipo: Digital, Tradicional, Producto]

## Canales que manejo
- [ ] Instagram
- [ ] LinkedIn
- [ ] Facebook
- [ ] TikTok
- [ ] Email
- [ ] Website

## Especialización
- [ ] Estrategia de contenido
- [ ] Publicidad pagada
- [ ] SEO
- [ ] Email marketing
- [ ] Análisis de datos

## Mi audiencia
[Describe tu cliente ideal]
```

Paso 4: Integrar con OpenCode

```
# Reiniciar OpenCode para cargar el nuevo perfil
opencode

# Dentro de OpenCode, verificar que reconoce tu perfil
/config
```

✅ Checklist de verificación

☐

- [ ] Archivo `~/.config/opencode/AGENTS.md` existe
- [ ] Contiene tu información real
- [ ] Tono y formato reflejados
- [ ] OpenCode lo reconoce

📸 Entregable

Captura de tu AGENTS.md personalizado.

LAB 3: Tu Primer Agente Especializado (25 minutos)

Objetivo

Crear un agente personalizado para tu caso de uso específico.

Paso 1: Crear directorio de agentes

```
mkdir -p ~/.config/opencode/agents
```

Paso 2: Template de Agente

Crea `~/.config/opencode/agents/mi-agente.md`:

```
---
name: mi-agente
description: "Trigger: [keyword1], [keyword2], [keyword3].
        [Descripción breve]"
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: allow
---

# Eres [describe quién es el agente]

## Tu rol

[Describe el rol específico]

## Áreas de expertise

1. [Área 1]
```

```
2. [Área 2]
3. [Área 3]

## Cómo comunicas

**Tono:** [Formal/Casual/Profesional]
**Estilo:** [Descriptivo/Técnico/Ejemplos]
**Idioma:** [Español/English/Ambos]

## Metodología de trabajo

### Para [tipo de tarea 1]
1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

### Para [tipo de tarea 2]
1. [Paso 1]
2. [Paso 2]

## Formatos de entrega

### Documento largo
```

[Título]

Resumen

[Breve resumen]

Contenido principal

[Detalle]

Conclusión

[Resumen final]

```
### Respuesta corta
```

[Punto principal] [Explicación breve] [Ejemplo si aplica]

```
## Restricciones

- ❌ [Nunca hacer esto]
- ❌ [Nunca hacer aquello]
- ✅ [Siempre hacer esto]
```

```
## Frases típicas de cierre

- "Quedo a su disposición para cualquier duda"
- "¿Hay algo más en lo que pueda ayudarle?"
- "Será un placer ampliar esta información"
```

Paso 3: Agentes por profesión

Agente para Educator:

```
---
name: educator
description: "Trigger: lesson, class, teaching, student,
        education. Asistente para educadores y docentes"
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: allow
---

# Eres un asistente para educadores

## Tu expertise

- Diseño curricular y planes de clase
- Creación de evaluaciones y rúbricas
- Estrategias didácticas
- Adaptación para diferentes estilos de aprendizaje
- Recursos educativos

## Tipos de contenido que generas

### Plan de Clase

PLAN DE CLASE

Título: [Tema] Duración: [X] minutos Curso: [Grado/Nivel] Objetivo: [Objetivo de aprendizaje]

MATERIALES: - [Material 1] - [Material 2]

DESARROLLO: 1. Inicio ([X] min): [Actividad] 2. Desarrollo ([X] min): [Actividad] 3. Cierre ([X] min): [Actividad]

EVALUACIÓN: [Tipo de evaluación] [Rúbrica si aplica]


### Evaluación

EVALUACIÓN
```

Curso: [Grado/Nivel] Materia: [Nombre] Fecha: [Fecha] Duración: [X] minutos

INSTRUCCIONES: [Instrucciones claras]

PREGUNTAS:

I. [Tipo: opción múltiple, desarrollo, etc.] [Puntaje]

1. [Siguiente pregunta] [Puntaje]

CUADRO DE RESPUESTAS 1. [Opciones] 2. 3.

Rúbrica

RÚBRICA DE EVALUACIÓN

Criterio	Excelente (4)	Bueno (3)	Suficiente (2)	Insuficiente (1)
[Criterio 1]	[Desc]	[Desc]	[Desc]	[Desc]
[Criterio 2]	[Desc]	[Desc]	[Desc]	[Desc]

Agente para Administrador:

```
---
name: admin
description: "Trigger: report, analysis, meeting, planning, admin.
        Asistente para administradores"
mode: subagent
permission:
  edit: allow
  bash: deny
  webfetch: allow
---

# Eres un asistente administrativo

## Tu expertise

- Reportes ejecutivos
- Análisis de datos
- Minutas de reunión
- Planificación
- Presentaciones ejecutivas

## Tipos de documentos

### Reporte Ejecutivo

REPORTE EJECUTIVO
```

Fecha: [Fecha] Preparado por: [Tu nombre] Para: [Audiencia objetivo]

RESUMEN EJECUTIVO [2-3 oraciones del punto más importante]

HALLAZGOS PRINCIPALES 1. [HECHO]: [IMPLICACIÓN] 2. [HECHO]: [IMPLICACIÓN]

ANÁLISIS [Análisis de 2-3 párrafos]

RECOMENDACIONES 1. [Recomendación 1] → Impacto: [Alto/Medio/Bajo] 2. [Recomendación 2] → Impacto: [Alto/Medio/Bajo]

PRÓXIMOS PASOS - [ ] [Acción 1] - Fecha límite: [Fecha] - [ ] [Acción 2] - Fecha límite: [Fecha]

Minuta de Reunión

MINUTA DE REUNIÓN

Fecha: [Fecha] Hora: [Hora inicio] - [Hora fin] Lugar/Virtual: [Ubicación/Plataforma] Asistentes: [Lista de nombres] Ausentes: [Lista si aplica]

OBJETIVO: [Propósito de la reunión]

AGENDA: 1. [Tema 1] - [Responsable] - [Duración] 2. [Tema 2] - [Responsable] - [Duración]

DESARROLLO:

1. [Tema 1] Discusión: [Resumen de puntos discutidos] Acuerdos: [Lista de acuerdos]

2. [Tema 2] Discusión: Resumen Acuerdos: [Lista]

ACUERDOS GENERALES | Acuerdo | Responsable | Fecha límite | |———|————-|————| | [Acuerdo 1] | [Nombre] | [Fecha] | | [Acuerdo 2] | [Nombre] | [Fecha] |

PRÓXIMA REUNIÓN Fecha: [Fecha si está definida]

Checklist de Verificación
- [ ] Agente creado en `~/.config/opencode/agents/`
- [ ] Archivo .md con frontmatter correcto
- [ ] Descripción con keywords específicas
- [ ] Permissions configuradas

LAB 4: Conectar Extensiones MCP (20 minutos)

Objetivo
Conectar OpenCode con herramientas externas que uses.

Paso 1: Instalar Context7 (Documentación)

Context7 permite buscar documentación actualizada de librerías.

```bash
# Añadir MCP de Context7
opencode mcp add

# Seleccionar:
# Nombre: context7
# Tipo: Remote
# URL: https://mcp.context7.com/mcp

# O manualmente editar ~/.config/opencode/mcp.json
```

Configuración manual en `~/.config/opencode/mcp.json`:

```json
{
  "mcp": {
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "enabled": true
    }
  }
}
```

Paso 2: Configurar GitHub MCP (Opcional)

```bash
# Añadir GitHub MCP
opencode mcp add

# Seleccionar:
# Nombre: github
# Tipo: Local
# Comando: npx
# Args: -y @modelcontextprotocol/server-github
```

Paso 3: Verificar MCPs instalados

```bash
# Listar MCPs activos
opencode mcp list

# Ver detalles
opencode mcp ls
```

Paso 4: Usar Context7

```bash
opencode
```

```
# Dentro de OpenCode:
@general Buscame la documentación actualizada de React para hacer
        un formulario con validación
```

MCPs recomendados por profesión

Profesión	MCP recomendado
Educator	NotebookLM, Context7
Administrador	GitHub, Jira
Marketing	Slack, Notion
Programador	GitHub, PostgreSQL
Legal	Notion, Filesystem
Investigador	NotebookLM, ArXiv

✅ Checklist de verificación

- [ ] Al menos 1 MCP instalado
- [ ] `opencode mcp list` muestra el MCP
- [ ] El MCP responde correctamente

📸 Entregable

Captura de `opencode mcp list` mostrando tu MCP instalado.

LAB 5: Caso Práctico End-to-End (30 minutos)

Objetivo

Completar una tarea real usando todo lo aprendido.

Escenario: Educator

Contexto: Eres profesor/a de historia y mañana tienes una clase sobre la Revolución Francesa. Necesitas preparar el material en 30 minutos.

Tarea: Usando OpenCode, prepara: 1. Plan de clase (90 minutos) 2. Presentación de 10 slides 3. Evaluación de 5 preguntas

Instrucciones paso a paso

1. Abre OpenCode

```
opencode
```

2. Carga tu perfil

```
/load @agents/educator
```

3. Genera el plan de clase

```
Necesito un plan de clase de 90 minutos sobre la Revolución
Francesa para estudiantes de 4to año de bachillerato.

Incluye:
- Objetivo de aprendizaje
- 3 actividades principales
- Recursos necesarios
- Evaluación formativa
```

4. Genera la presentación

```
Basado en el plan anterior, crea una presentación de 10 slides en
formato markdown.
Cada slide debe tener:
- Título
- 3-4 puntos clave
- Sugerencia visual entre paréntesis
```

5. Genera la evaluación

```
Crea una evaluación de 5 preguntas:
- 2 de opción múltiple
- 1 de respuesta corta
- 1 de desarrollo
- 1 de análisis de documento

Incluye rúbrica de corrección.
```

Escenario: Administrador

Contexto: Eres gerente de operaciones y necesitas presentar un informe trimestral a la junta directiva.

Tarea: Preparar el reporte ejecutivo completo.

Instrucciones

```
opencode

/load @agents/admin

Prepara un reporte ejecutivo trimestral con:

1. Resumen ejecutivo (150 palabras)
2. Métricas clave del trimestre:
   - Ventas: $X (vs meta $Y)
   - Clientes nuevos: Z
   - Retención: W%
3. Análisis de tendencias
4. 3 principales logros
5. 2 principales desafíos
6. Plan para el próximo trimestre
7. Presupuesto solicitado

Usa formato ejecutivo profesional.
```

Escenario: Marketing

Contexto: Eres coordinador de marketing y necesitas planificar el contenido de Instagram para la próxima semana.

Instrucciones

```
opencode

/load @agents/marketing

Crea un calendario de contenido para Instagram para la próxima
semana (7 días).

Para cada día incluye:
- Tipo de post (educativo, motivacional, producto, interactivo)
- Tema/hook
- Caption (máximo 150 caracteres)
- 5 hashtags relevantes
- Mejor hora de publicación
- Call-to-action

Considera que somos una pyme de [tu industria] en Ecuador.
```

✅ Checklist de verificación

- [ ] Plan de clase creado y completo
- [ ] Presentación con estructura lógica

- [ ] Evaluación con rúbrica
- [ ] Contenido exportado/guardado

📸 Entregable

Carpeta con: - Plan de clase.md - Presentacion.md - Evaluacion.md

---

Lab Bonus: Automatización con Skills

Crear un skill personalizado

```
mkdir -p ~/.config/opencode/skills/mi-skill
```

~/.config/opencode/skills/mi-skill/SKILL.md:

```
---
name: mi-skill
description: "Trigger: [tu keyword]. [Descripción]"
---

# Mi Skill Personal

## Cuándo usarlo
[Describe cuándo activar este skill]

## Proceso
1. [Paso 1]
2. [Paso 2]
3. [Paso 3]

## Template

[TU TEMPLATE AQUÍ]


## Checklist
- [ ] [Verificación 1]
- [ ] [Verificación 2]
```

Usar el skill

```
opencode

# Activar
@general /load skills/mi-skill
```

```
# Usar
[Tu keyword]: [tarea]
```

Resumen de Labs

Lab	Tema	Duración	Entregable
1	Instalación	15 min	Screenshot versión
2	Perfil	20 min	AGENTS.md personalizado
3	Agente	25 min	1 agente especializado
4	MCPs	20 min	1 MCP configurado
5	E2E	30 min	Material completo

Tiempo total: ~110 minutos (con pausa incluida)

Recursos Adicionales

Documentación

- docs.opencode.ai
- Awesome OpenCode

Comunidad

- Discord: OpenCode Community
- GitHub: anomalyco/opencode

Labs creados para el Taller de IA para No Informáticos Duración estimada: 2 horas incluyendo teoría